Operating Systems

[1500WETOPS]

José Oramas

Cache Memory

[Speeding up Access to Memory]

José Oramas

University of Antwerp

Announcement

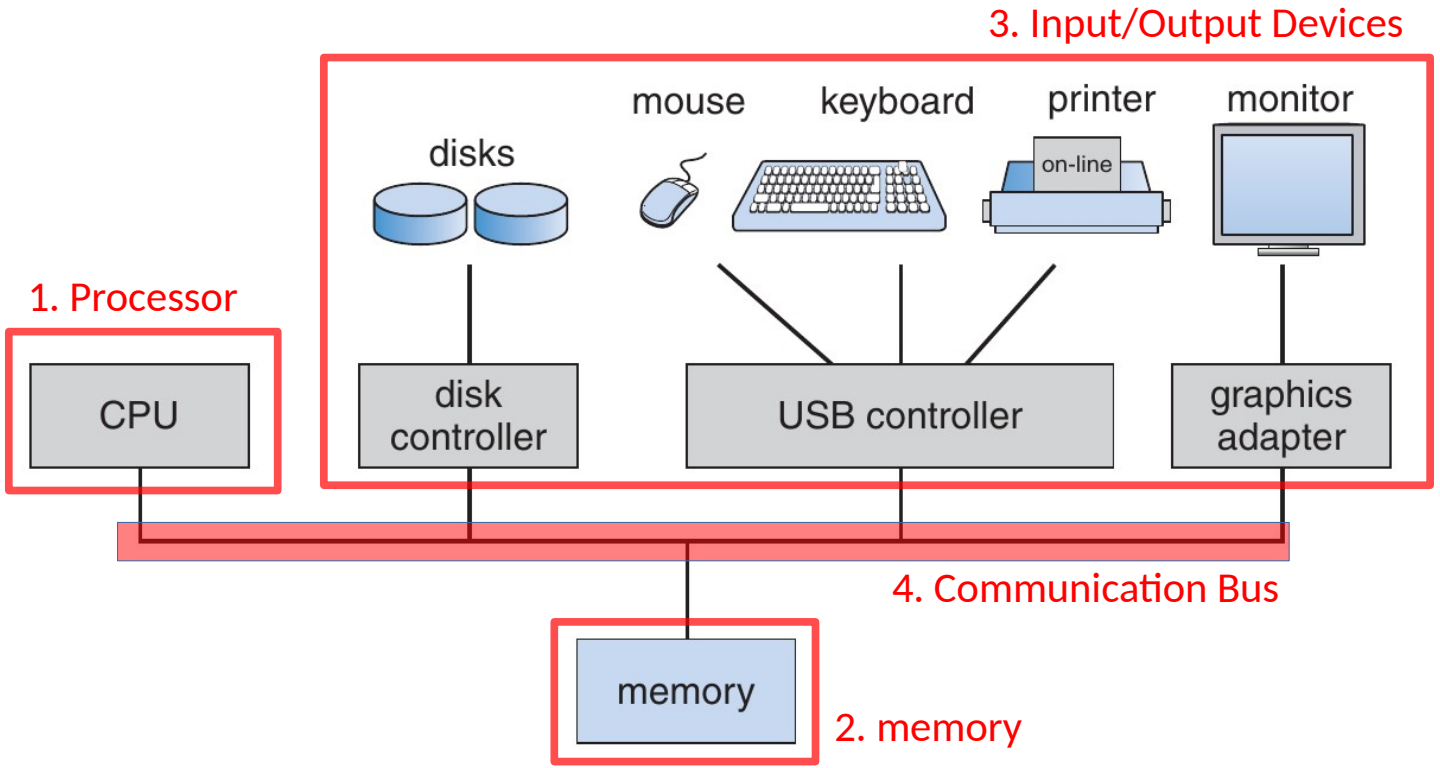
Exercise Session #1

- Tomorrow 16h00-18h00, M.G.005
- Caching + benchmarking

Storage: background

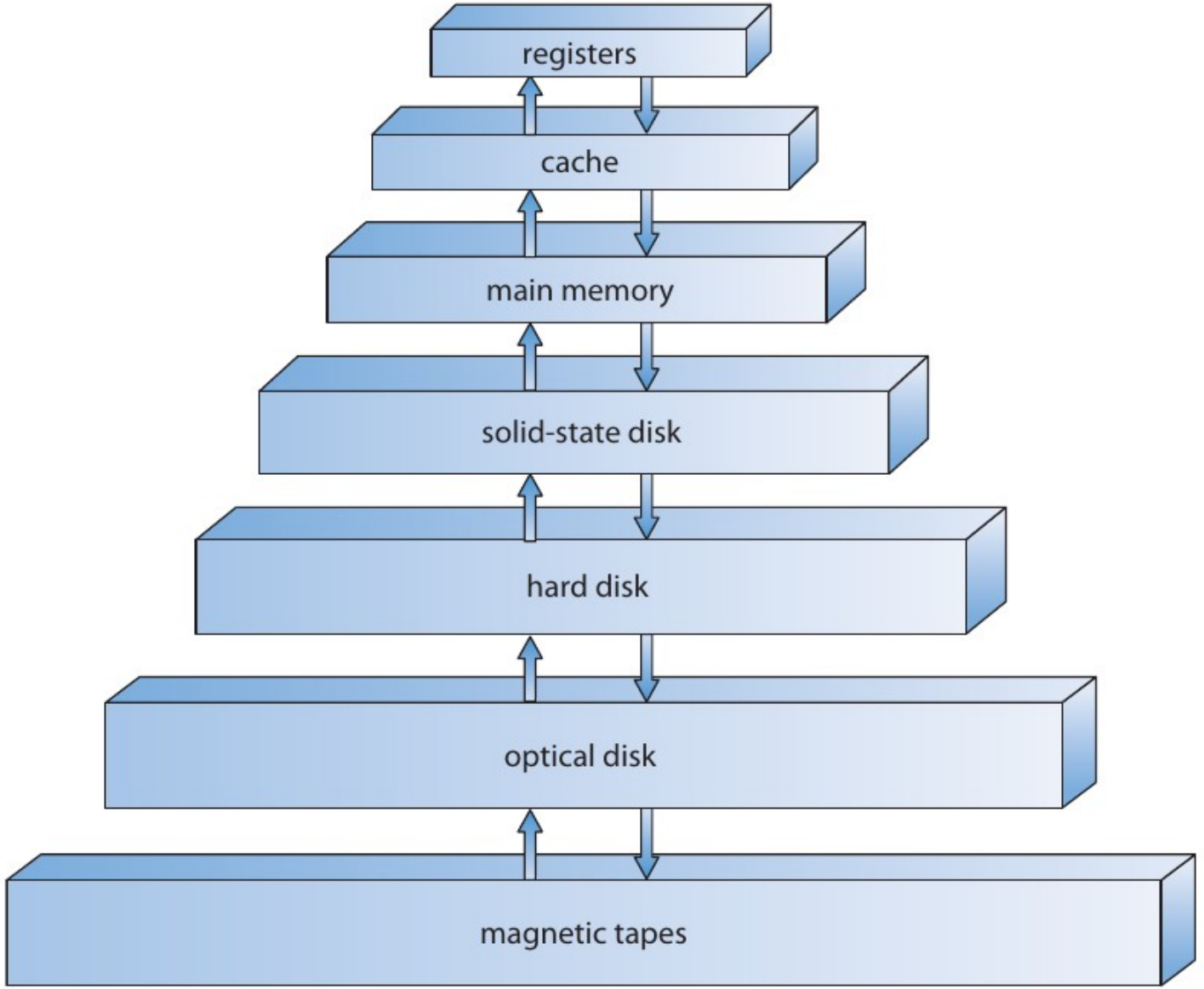
Different Types of Memory

- With different capacities and access speed
- Discrepancy between processor and storage access speed.



Storage: background

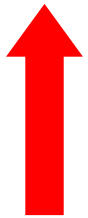
Storage Hierarchy



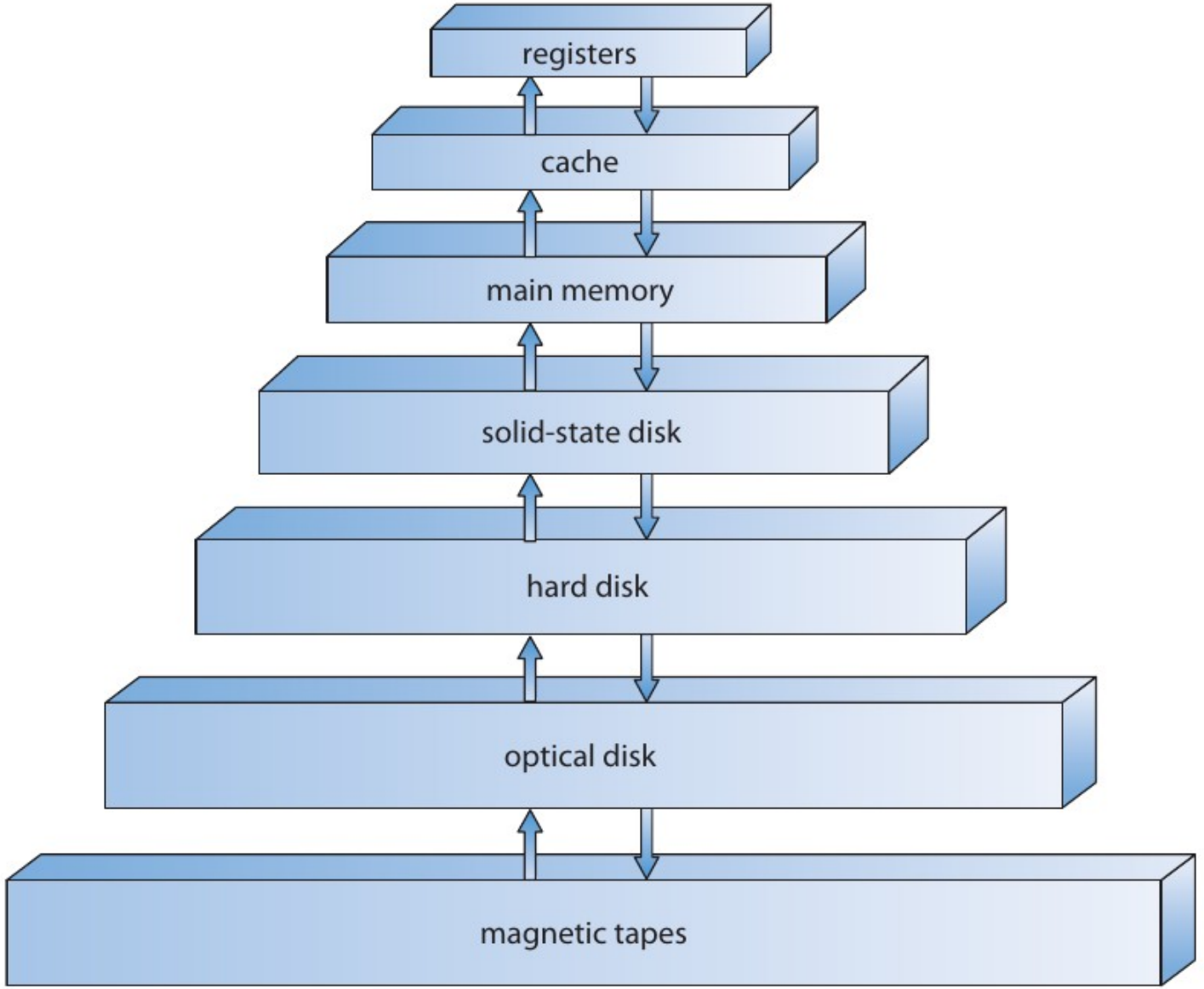
Storage: background

Storage Hierarchy

+ cost
+ speed



+ capacity

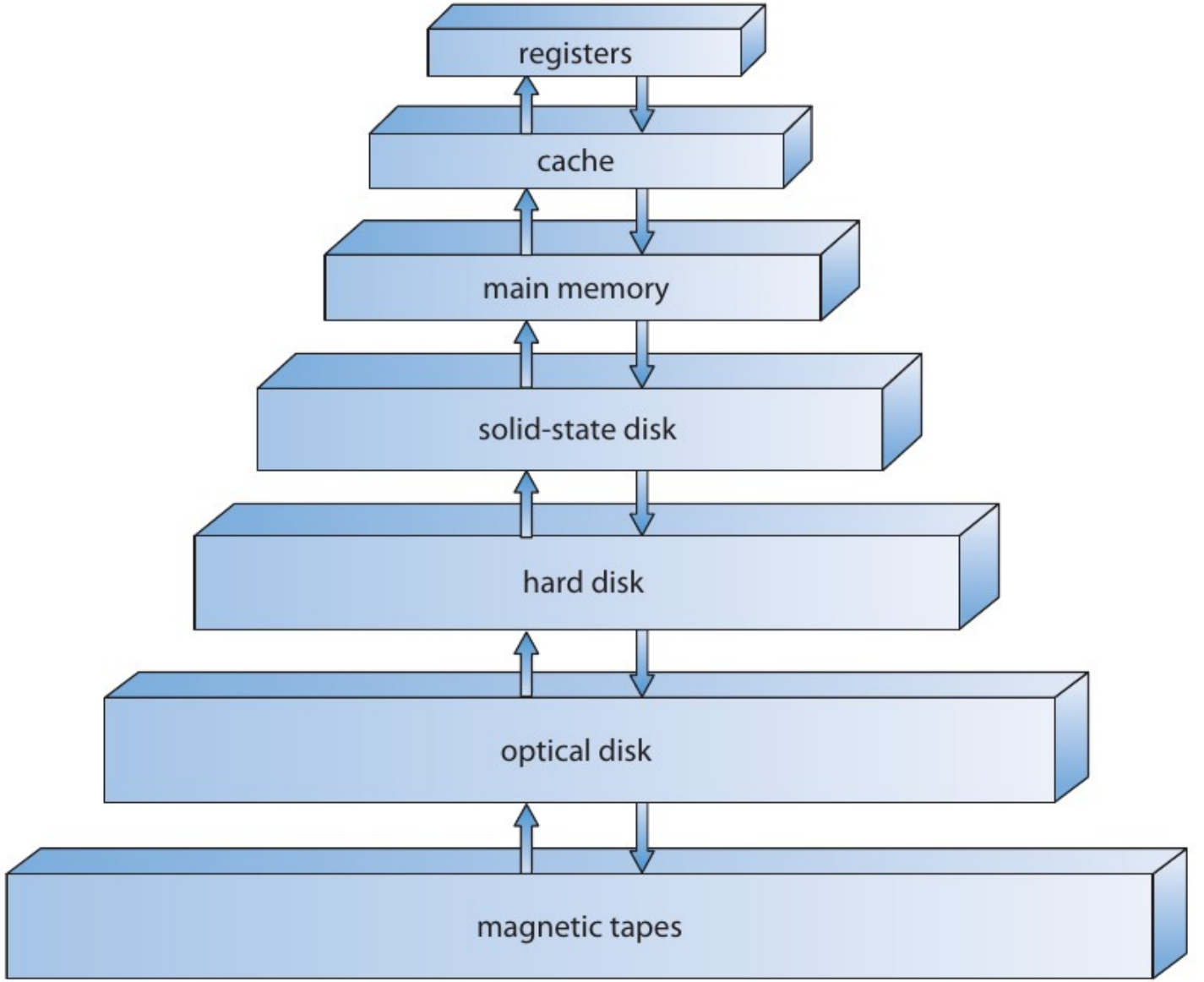


Storage: background

Storage Hierarchy

+ cost
+ speed

+ capacity

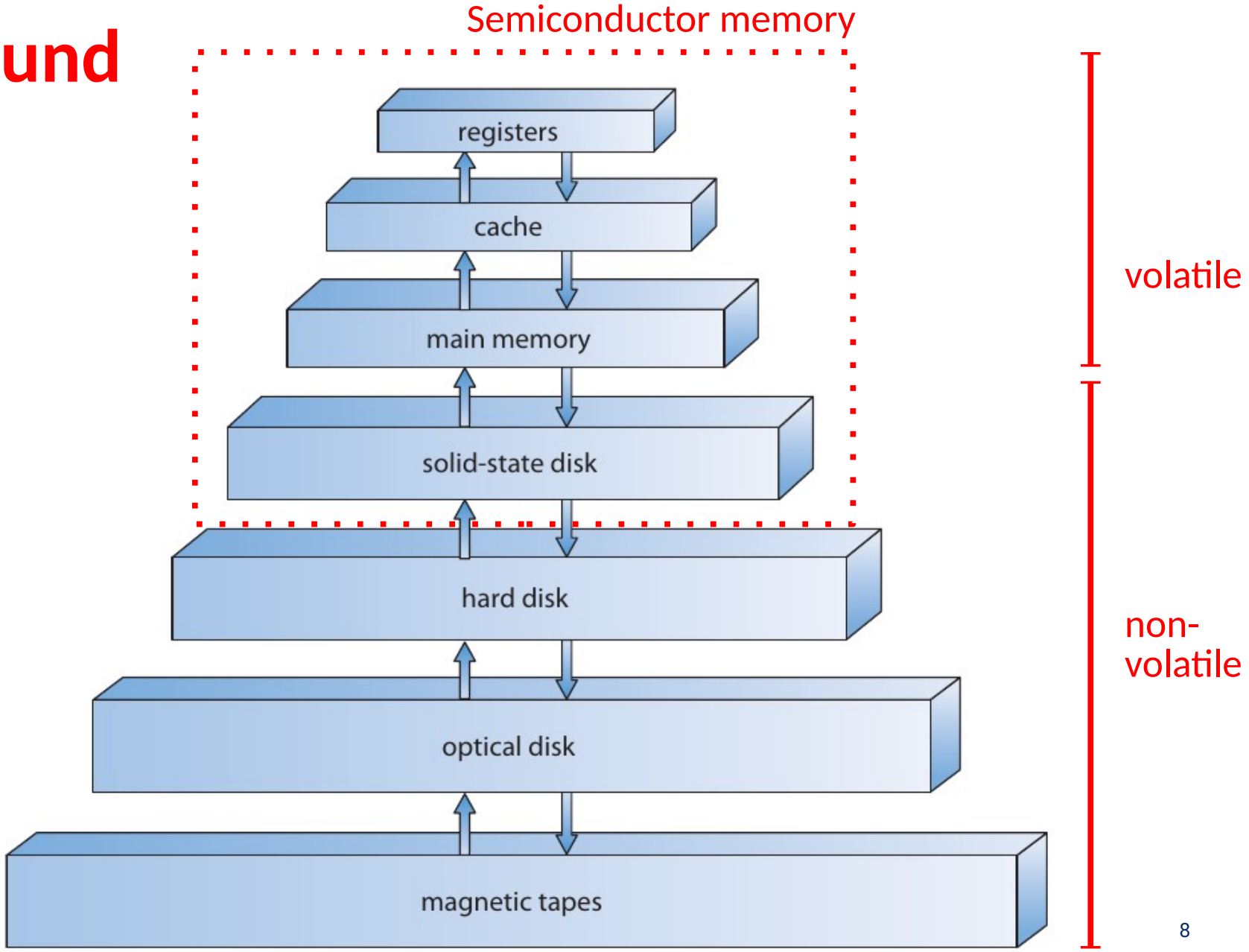


Storage: background

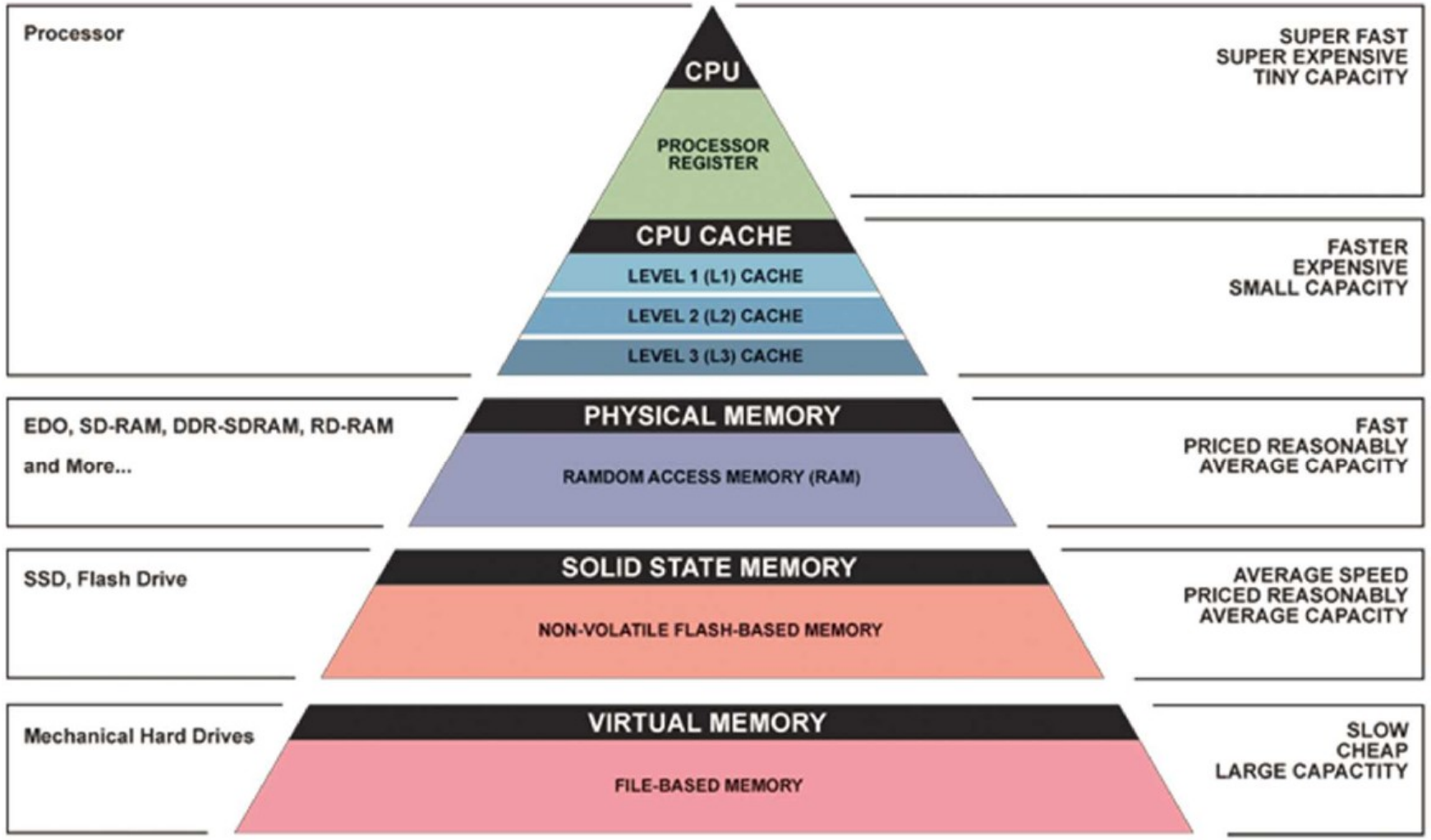
Storage Hierarchy

+ cost
+ speed

+ capacity



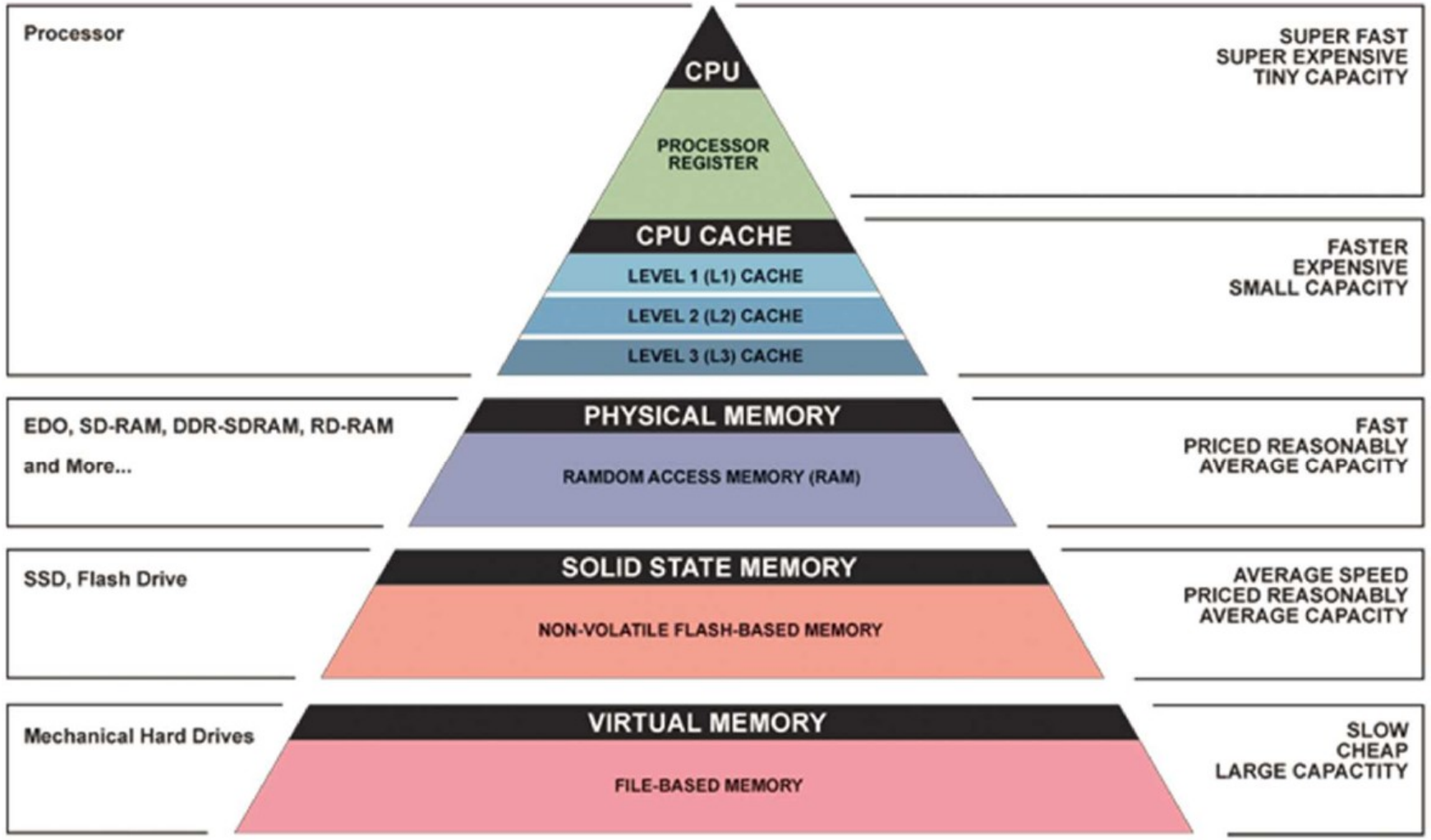
Memory Hierarchy



Note

- Capacity, speed, cost relationships still in place

Memory Hierarchy



Note

- Capacity, speed, cost relationships still in place
- How to make these work in practice?
- Why not select only the fastest components?

Memory Technology Evolution

SRAM

Metric	1980	1985	1990	1995	2000	2005	2010	*2010:1980*
$/MB	19,200	2,900	320	256	100	75	60	*320*
access (ns)	300	150	35	15	3	2	1.5	*200*

Memory Technology Evolution

SRAM

Metric	1980	1985	1990	1995	2000	2005	2010	*2010:1980*
$/MB	19,200	2,900	320	256	100	75	60	*320*
access (ns)	300	150	35	15	3	2	1.5	*200*

DRAM

Metric	1980	1985	1990	1995	2000	2005	2010	*2010:1980*
$/MB	8,000	880	100	30	1	0.1	0.06	*130,000*
access (ns)	375	200	100	70	60	50	40	*9*
typical size (MB)	0.064	0.256	4	16	64	2,000	8,000	*125,000*

Memory Technology Evolution

SRAM

Metric	1980	1985	1990	1995	2000	2005	2010	*2010:1980*
$/MB	19,200	2,900	320	256	100	75	60	*320*
access (ns)	300	150	35	15	3	2	1.5	*200*

DRAM

Metric	1980	1985	1990	1995	2000	2005	2010	*2010:1980*
$/MB	8,000	880	100	30	1	0.1	0.06	*130,000*
access (ns)	375	200	100	70	60	50	40	*9*
typical size (MB)	0.064	0.256	4	16	64	2,000	8,000	*125,000*

Disk

Metric	1980	1985	1990	1995	2000	2005	2010	*2010:1980*
$/MB	500	100	8	0.30	0.01	0.005	0.0003	*1,600,000*
access (ms)	87	75	28	10	8	*4*	*3*	*29*
typical size (MB)	1	10	160	1,000	20,000	160,000	1,500,000	*1,500,000*

- SRAM used for cache, DRAM for main memory

Memory

Locality of Reference

- The hierarchy works when:
  - Data Request Frequency drops, significatively, as we go lower in the pyramid.

Memory

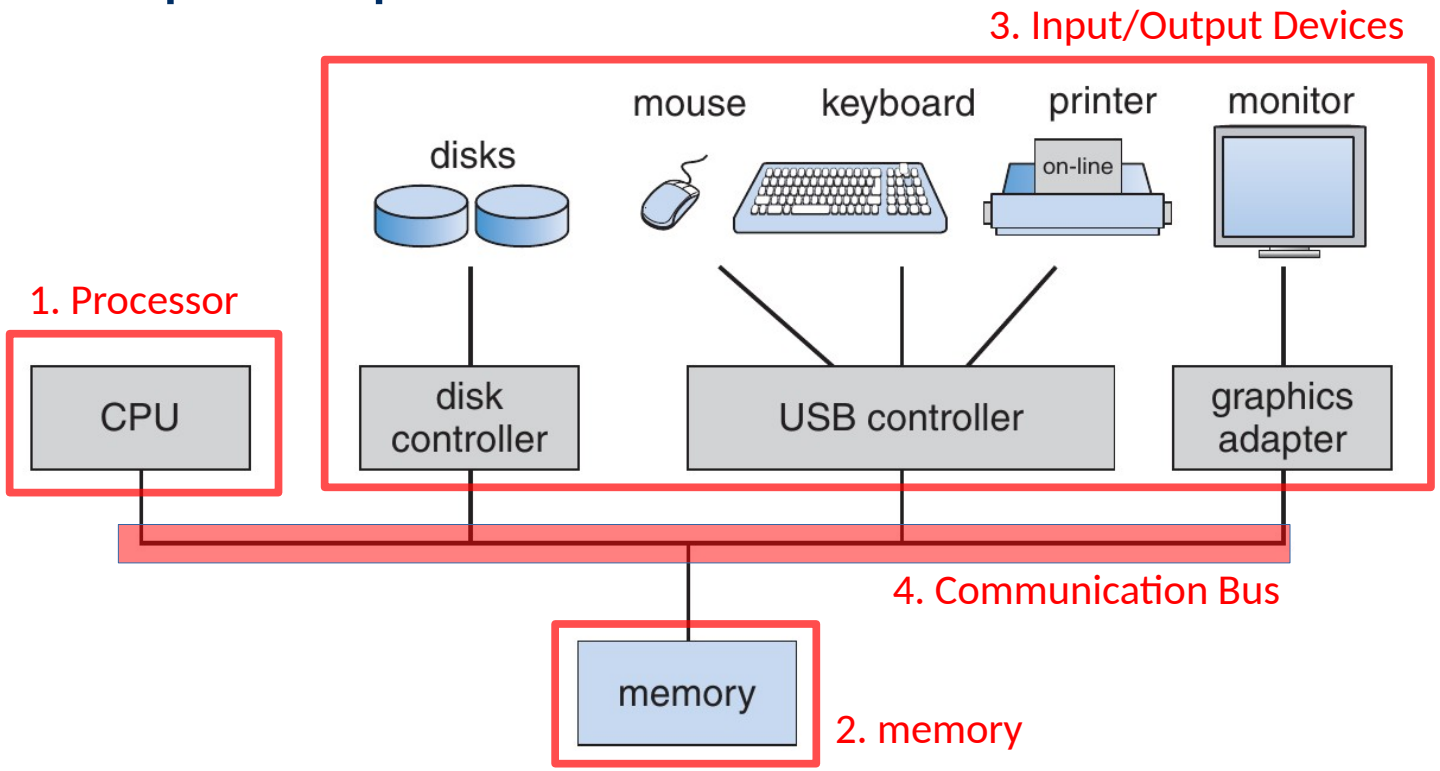
Locality of Reference

- The hierarchy works when:
  - Data Request Frequency drops, significatively, as we go lower in the pyramid.
- This follows the locality of reference principle:
  - Spatial locality (PC, arrays, etc.)
  - Temporal locality (variables, for/while loops, etc.

Memory

Cache connection

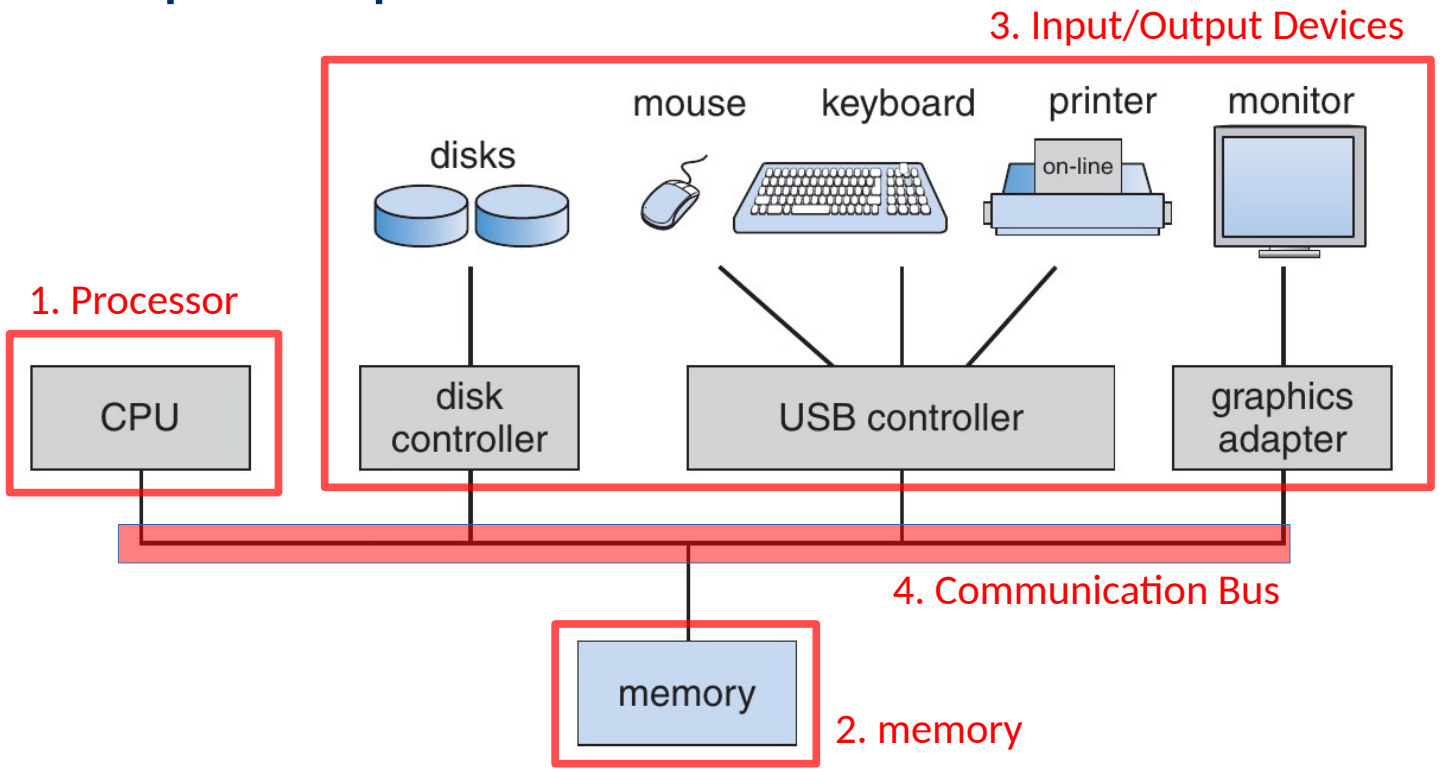
- Use cache as an intermediate component between memory and CPU to speed up access



Memory

Cache connection

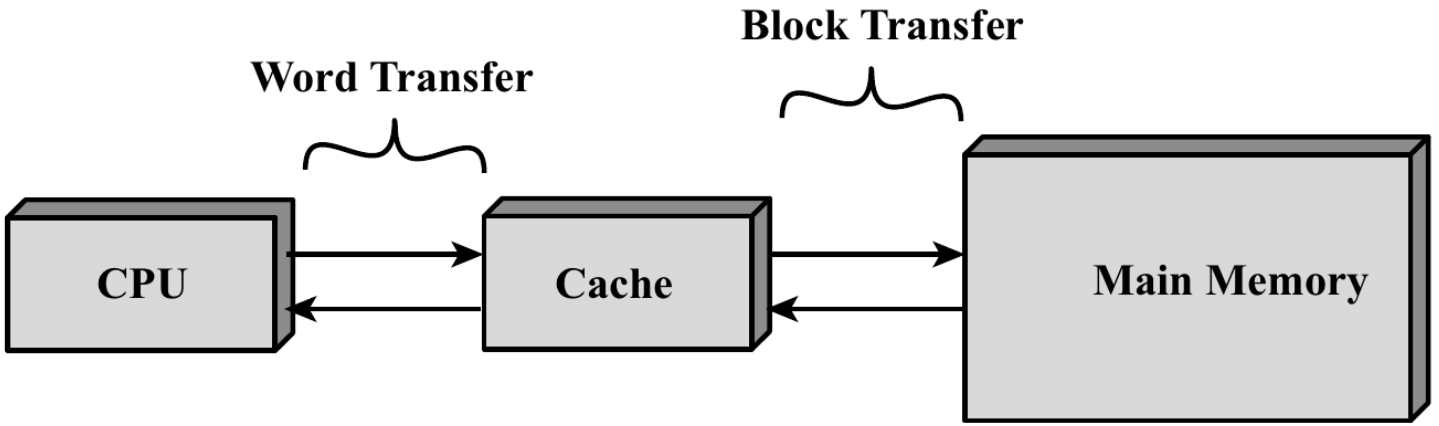
- Use cache as an intermediate component between memory and CPU to speed up access



What about connecting the cache via the standard bus (4) ?

Memory

Cache Connection

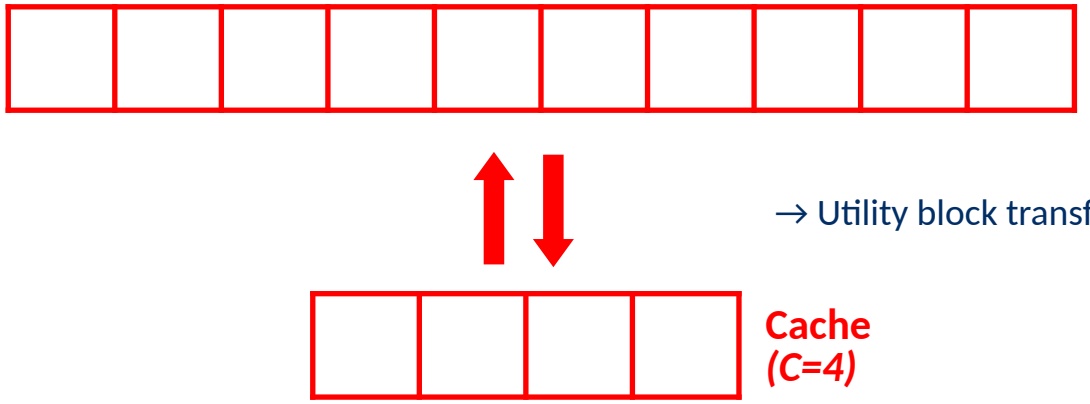


- Fast bus (on-chip) between (L1) cache and the processor (CPU).
  - Transfer unit → *word*
- Slower shared bus between cache and the main memory
  - block transfer → cache line

Cache Memory

Organization

- Memory is partioned in ***M*** blocks, each containing ***K*** words.
- A cache containing ***C*** lines (contain *C* blocks) ; *C* << *M*
  - Transfer pushes blocks, i.e. several words at a time
- Each line has a **tag** used to identify the block it refers to.
  - Also a bit to indicate whether the line is in use

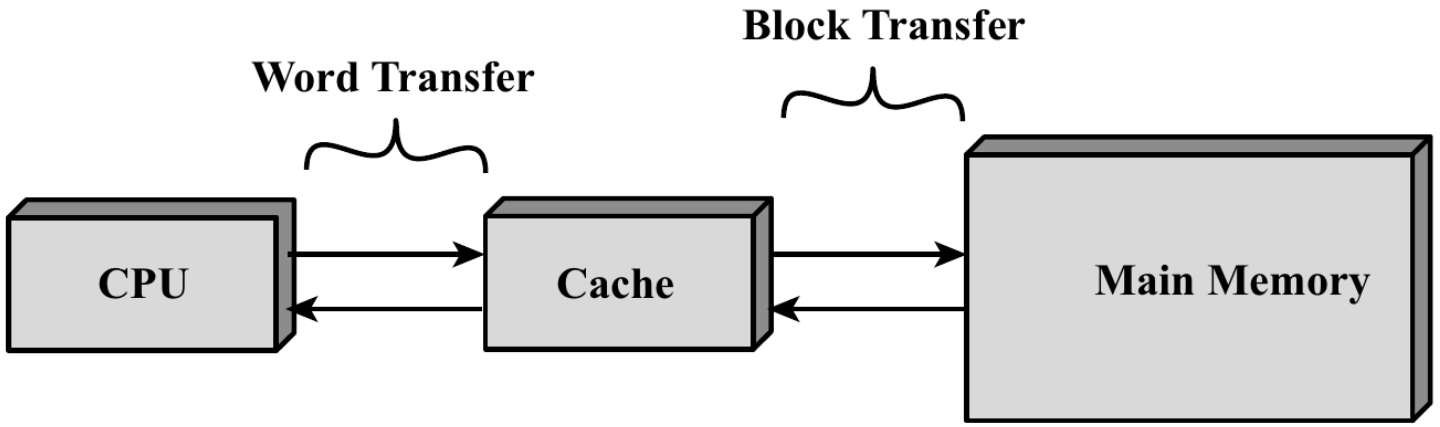


→ Utility block transfer: locality of reference & slower bus access

Cache Memory

Operation

- The processor requests to read a *particular* word from memory:



Note

- *default hit ratio H = C/M*

Effect of the block size?

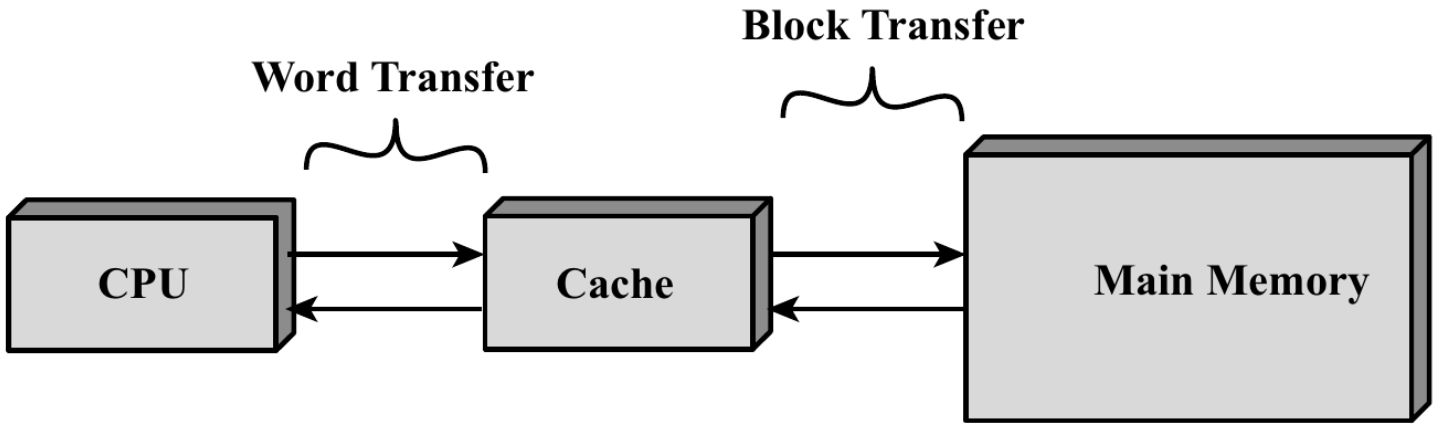
- **↓ block size,** ↑ lines, ↓effect of spatial locality
- **↑ block size,** ↓ lines, eventual ↓effect of spatial locality

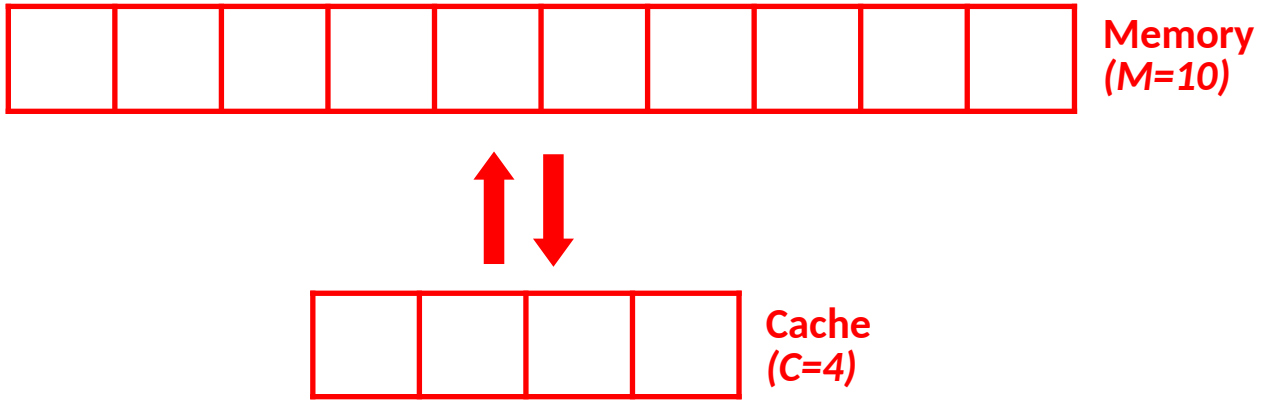
Mapping Functions

[linking main memory with cache]

Mapping Functions

Link a block *j* (from memory) with a line *i* in cache.





Mapping Functions

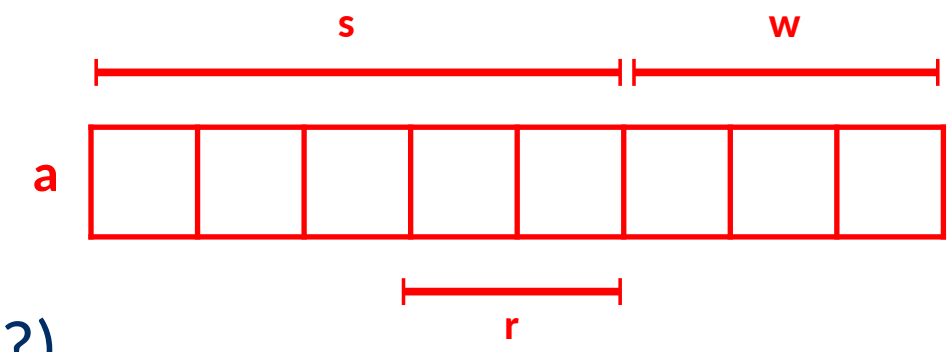
Direct Mapping $i = j \mod 2^r (= C)$

- Memory block (*j*) can only be placed in one place *(i)* in the cache
- Translation is simple
- Easy to look up

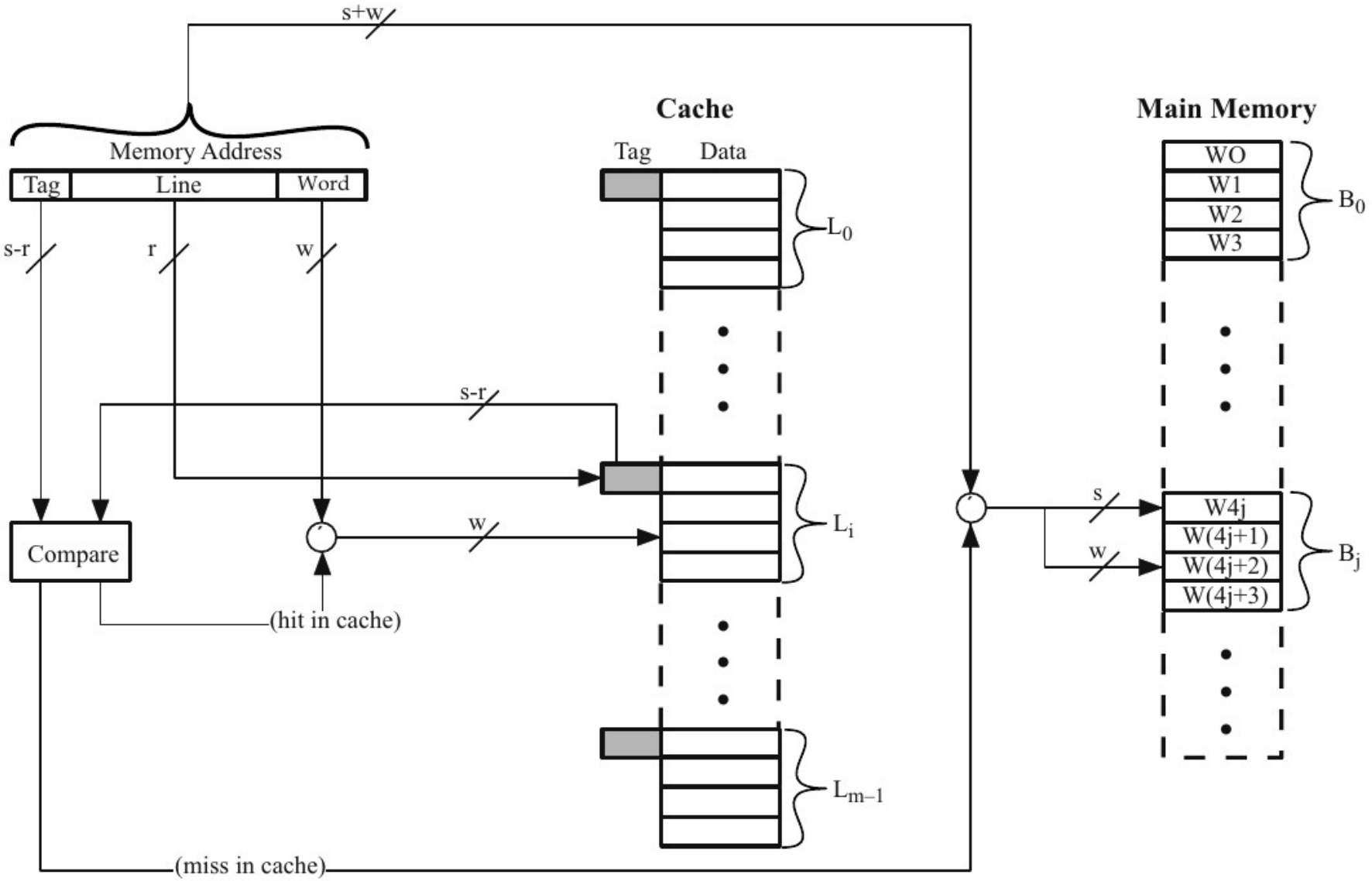
Mapping Functions

Direct Mapping $i = j \mod 2^r (= C)$

- Memory block (*j*) can only be placed in one place *(i)* in the cache
- Translation is simple
- Easy to look up
- Assume address consists of s+w bits
  - 2^w words per cache line
  - $C = 2^r$ lines in the cache
  - Tag of s-r bits needed (why first bits?)



Direct Mapping $i = j \mod 2^r (= C)$



Procedure

- Check the cache line *(r)*
- Compare the tag *(s-r)*
- Look for requested word *(w)*

Direct Mapping

Consider

- 4 Kbyte Cache
- Blocks of 32 bytes
- Float of 4 bytes → 8 floats/line
- Both ***a*** and ***b*** are not in cache

```
float a[1024], b[1024];

for (i=0; i<1024; i++)
  sum += a[i]*b[i];
```

i	Operation	Status	In cache	Comment
0	load a[0]	miss	a[0..7]	assume a[] was not cached yet
	load b[0]	miss	b[0..7]	assume b[] was not cached yet
	t=a[0]*b[0]			
	sum += t			
1	load a[1]	hit	a[0..7]	previous load brought it in
	load b[1]	hit	b[0..7]	previous load brought it in
	t=a[1]*b[1]			
	sum += t			
	..... etc			
7	load a[7]	hit	a[0..7]	previous load brought it in
	load b[7]	hit	b[0..7]	previous load brought it in
	t=a[7]*b[7]			
	sum += t			
8	load a[8]	miss	a[8..15]	this line was not cached yet
	load b[8]	miss	b[8..15]	this line was not cached yet
	t=a[8]*b[8]			
	sum += t			
9	load a[9]	hit	a[8..15]	previous load brought it in
	load b[9]	hit	b[8..15]	previous load brought it in
	t=a[9]*b[9]			
	sum += t			
	.....			

Direct Mapping

Consider

- 4 Kbyte Cache
- Blocks of 32 bytes
- Float of 4 bytes → 8 floats/line
- Both ***a*** and ***b*** are not in cache

→ hit rate: 87.5%

What if variables *a* and *b* are in memory, just right after the other?

```
float a[1024], b[1024];

for (i=0; i<1024; i++)
  sum += a[i]*b[i];
```

i	Operation	Status	In cache	Comment
0	load a[0]	miss	a[0..7]	assume a[] was not cached yet
	load b[0]	miss	b[0..7]	assume b[] was not cached yet
	t=a[0]*b[0]			
	sum += t			
1	load a[1]	hit	a[0..7]	previous load brought it in
	load b[1]	hit	b[0..7]	previous load brought it in
	t=a[1]*b[1]			
	sum += t			
	..... etc			
7	load a[7]	hit	a[0..7]	previous load brought it in
	load b[7]	hit	b[0..7]	previous load brought it in
	t=a[7]*b[7]			
	sum += t			
8	load a[8]	miss	a[8..15]	this line was not cached yet
	load b[8]	miss	b[8..15]	this line was not cached yet
	t=a[8]*b[8]			
	sum += t			
9	load a[9]	hit	a[8..15]	previous load brought it in
	load b[9]	hit	b[8..15]	previous load brought it in
	t=a[9]*b[9]			
	sum += t			
	.....			

Direct Mapping

Consider

- 4 Kbyte Cache
- Blocks of 32 bytes
- Float of 4 bytes → 8 floats/line
- Both ***a*** and ***b*** are not in cache

→ hit rate: 87.5%

What if variables *a* and *b* are in memory, just right after the other?

→ hit rate: 0%

```
float a[1024], b[1024];

for (i=0; i<1024; i++)
  sum += a[i]*b[i];
```

```
i   Operation Status In cache  Comment

0   load a[0] miss   a[0..7]   assume a[] was not cached yet
    load b[0] miss   b[0..7]   assume b[] was not cached yet
    t=a[0]*b[0]
    sum += t
1   load a[1] hit    a[0..7]   previous load brought it in
    load b[1] hit    b[0..7]   previous load brought it in
    t=a[1]*b[1]
    sum += t
    ..... etc .....
7   load a[7] hit     a[0..7]  previous load brought it in
    load b[7] hit     b[0..7]  previous load brought it in
    t=a[7]*b[7]
    sum += t
8   load a[8] miss    a[8..15] this line was not cached yet
    load b[8] miss    b[8..15] this line was not cached yet
    t=a[8]*b[8]
    sum += t
9   load a[9] hit     a[8..15] previous load brought it in
    load b[9] hit     b[8..15] previous load brought it in
    t=a[9]*b[9]
    sum += t
    .......
```

Associative Mapping

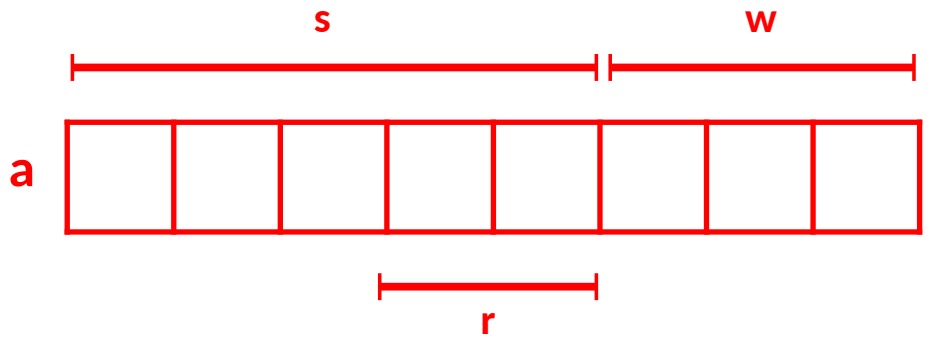
Description

- Allow blocks to be written anywhere in cache
  - We are free to choose where to write
- When a new word is requested:
  - Compare the tag of the requested word with all the tags in the cache (in parallel).

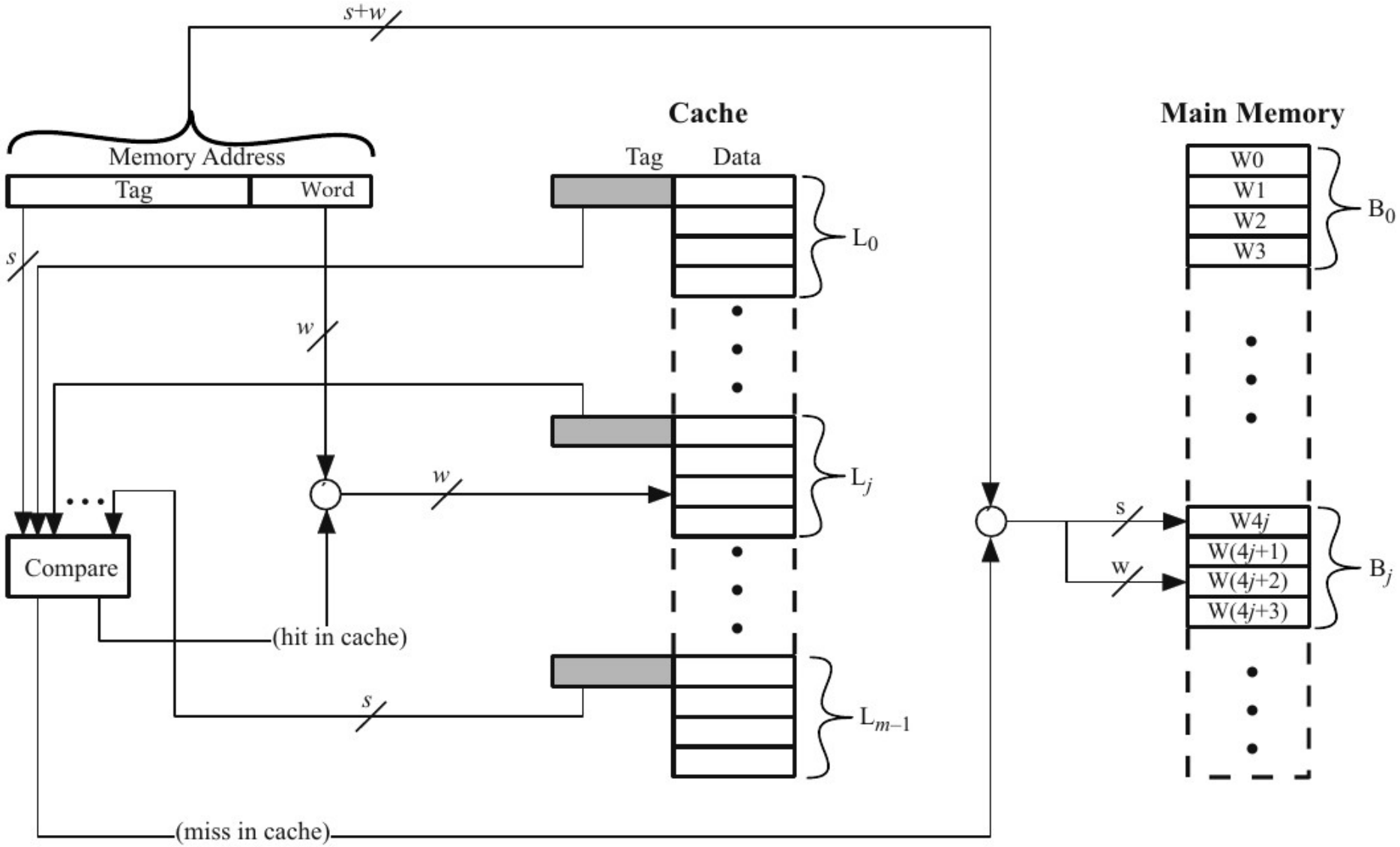
Associative Mapping

Description

- Allow blocks to be written anywhere in cache
  - We are free to choose where to write
- When a new word is requested:
  - Compare the tag of the requested word with all the tags in the cache (in parallel).
- Assume address consists of s+w bits
  - 2^w words per cache line
  - 2^r cache lines in the cache
  - Tag of s bits needed (why s bits?)



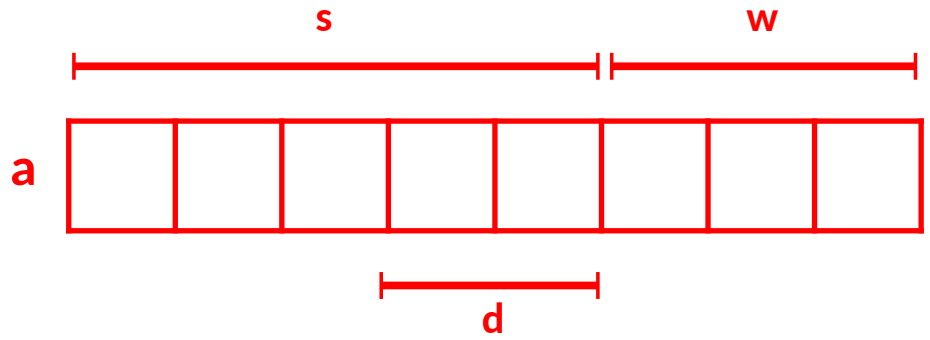
Associative Mapping



Procedure

- Compare the requested tag *(s)* with all in cache
- Look for requested word *(w)*

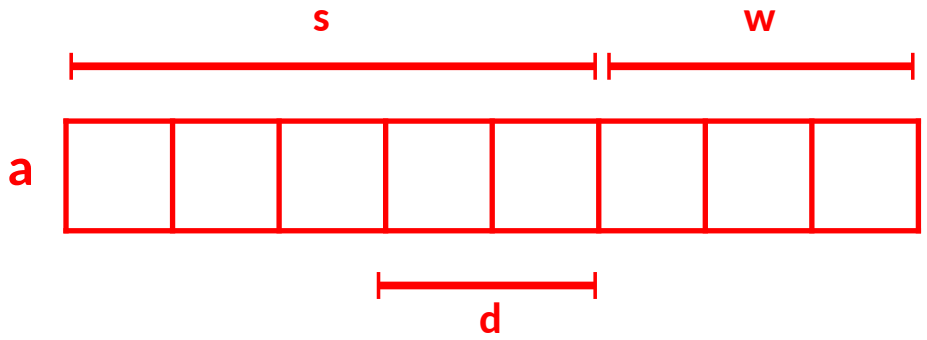
Set Associative Mapping



Description

- Divide the cache into $v=2^d$ sets, each containing $k = C/v = 2^{r-d}$ lines.
  - Each block in memory will be allowed to be located into only one of these sets.
  - Last ***d*** of the first ***s*** bits of an address refer to the set where a block could be located.
  - The tag (length *s-d*) of a word only needs to be compared to those of within the set where the word could be located

Set Associative Mapping

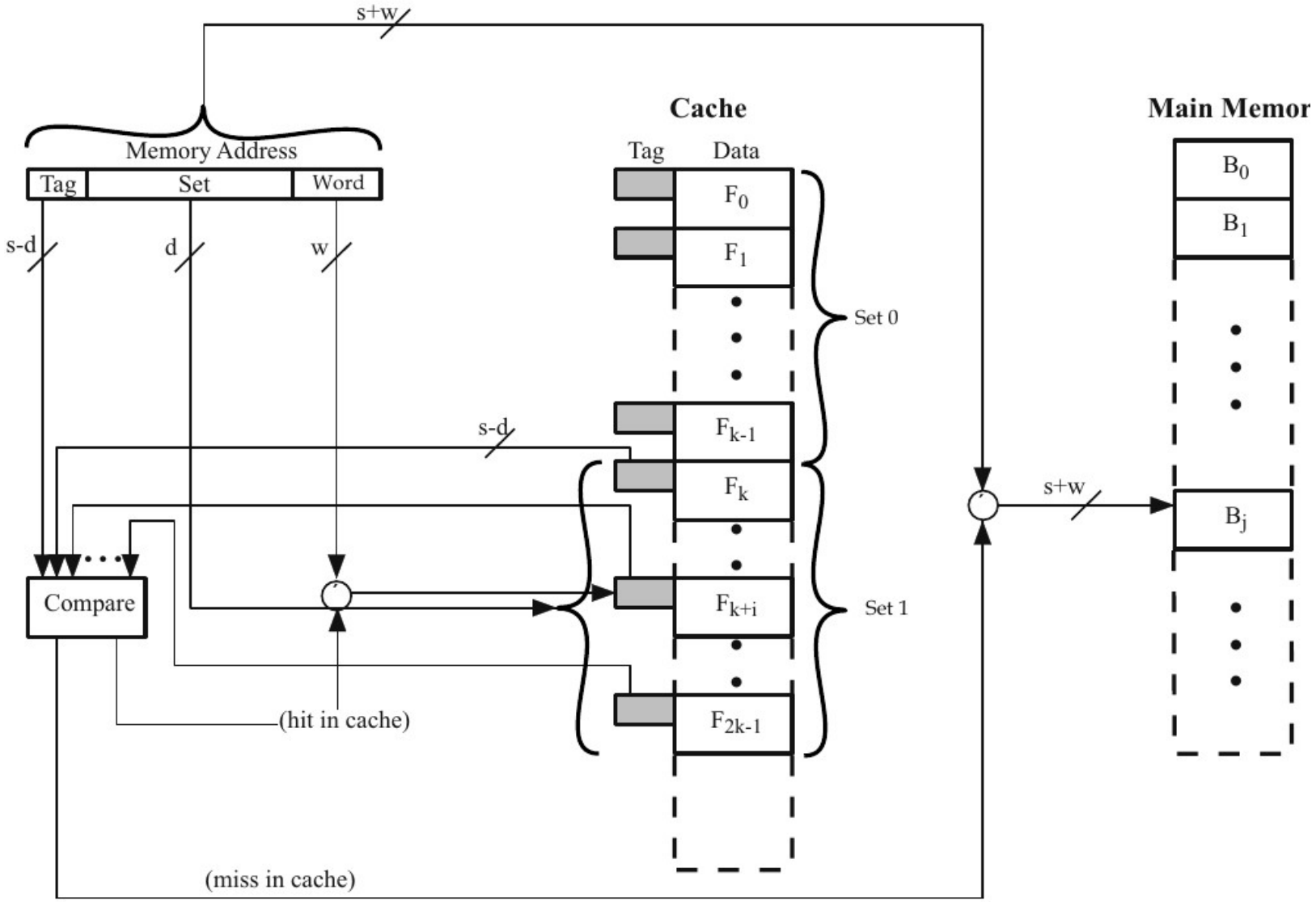


Description

- Divide the cache into $v=2^d$ sets, each containing $k = C/v = 2^{r-d}$ lines.
  - Each block in memory will be allowed to be located into only one of these sets.
  - Last ***d*** of the first ***s*** bits of an address refer to the set where a block could be located.
  - The tag (length *s-d*) of a word only needs to be compared to those of within the set where the word could be located
- k-way set associative cache.
  - if k==1 -> direct caching, if k==C -> associative caching

    (based on the number of tag comparisons)
  - k is usually very small and sufficient to reduce trashing.

Set Associative Mapping



Procedure

- Determine the set *(d)*
- Compare the tag *(s-d) with k tags in set.*
- Look for requested word *(w)*

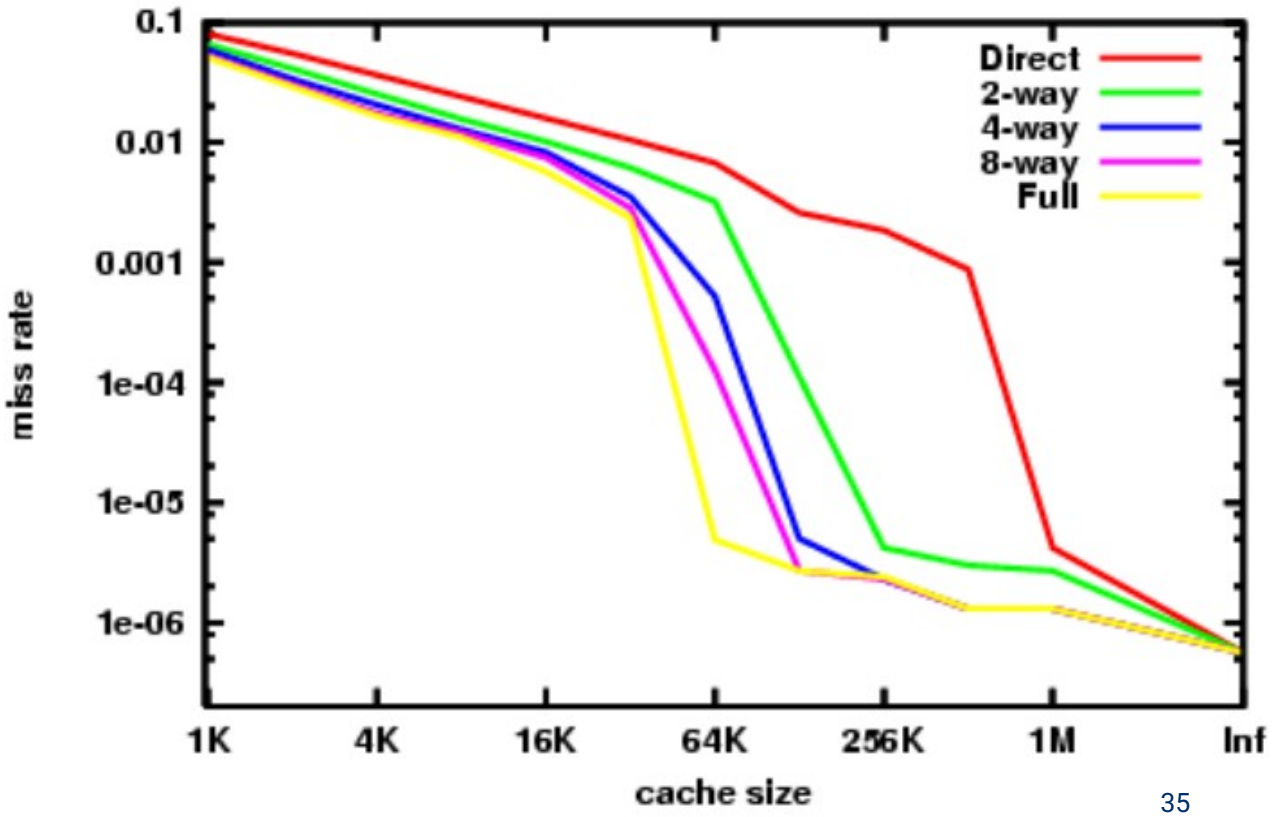
Set Associative Mapping

k-way set associative cache

- Based on the number of tag comparisons
  - if ***k==1*** → direct caching
  - if ***k==C*** → associative caching
- *k* is usually very small

  → sufficient to reduce trashing.

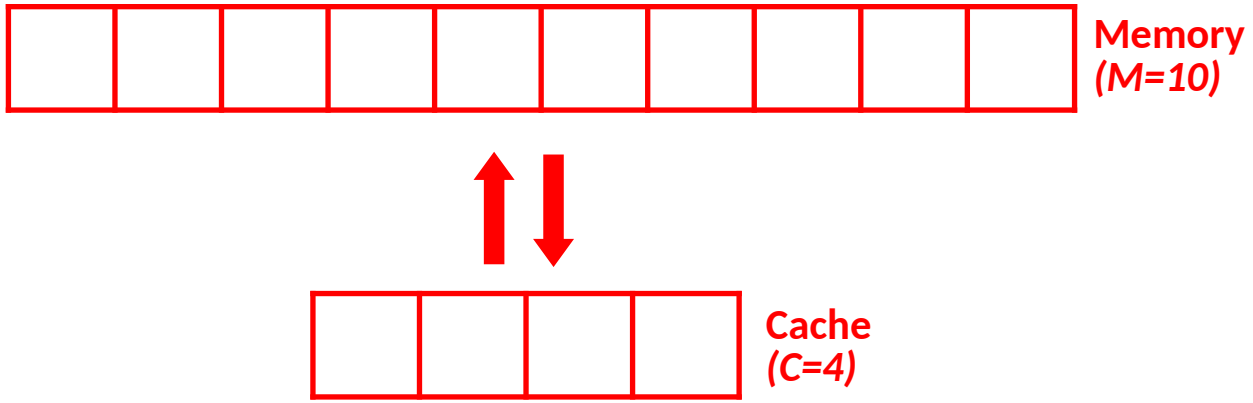
(PowerPC G3/4: 8-way, 2x 32 Kbyte)



Replacement Algorithms

[where to replace a value in cache]

Replacement Algorithms



Motivation

- For every cache miss
  - Put a new block in cache
  - With [Set-]Associative cache: choice
    (not applicable for direct mapping)
- Algorithm must be very efficient in hardware
- Optimal Algorithm
  - Replace lines that are the less likely needed in the future.

    (Not implementable in practice but used as reference)

Replacement Algorithms

Random

- Simple (no work at hit), not very intelligent, susceptible to thrashing
- Provable as worst case algorithm in certain context

Replacement Algorithms

Random

- Simple (no work at hit), not very intelligent, susceptible to thrashing
- Provable as worst case algorithm in certain context

FIFO: Replace the oldest line

- Simple (no work on hit), only one pointer required for implementation, less thrashing.
- Assumption: oldest lines are requested least frequently in the future

Replacement Algorithms

Least Recently Used (LRU)

- Replace the line that has been used least recently
- Provably best possible algorithm (in certain context)
- Often very effective against trashing
- Stack implementation:
  - k log2(k) bits per set
  - Also update with each cache hit (lots of work)
- Does not use frequency info

Replacement Algorithms

Pseudo LRU algorithms

- **Goal:** Performance from LRU + simplicity from FIFO
- Two variants:
  - Tree-based (TB) pseudo LRU
  - Most-recently-used (MRU) pseudo LRU

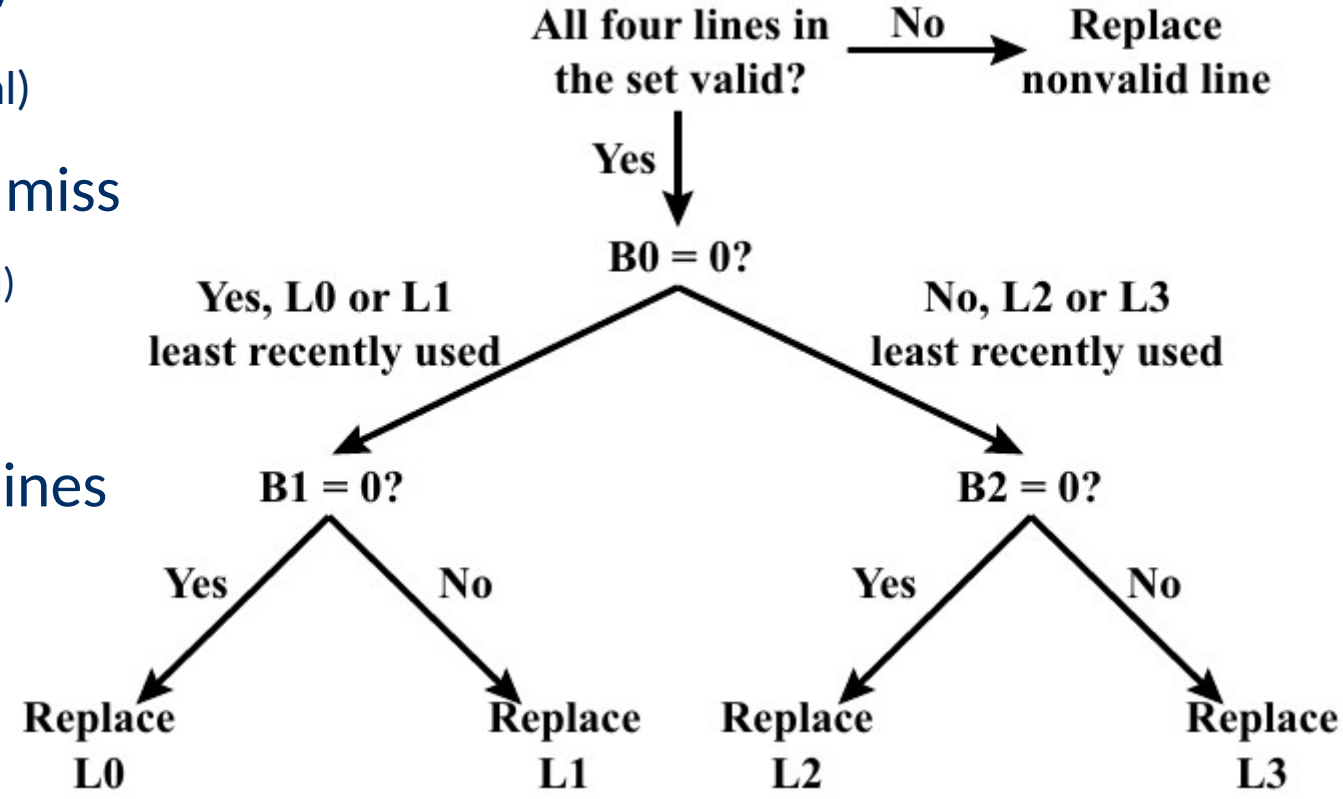
Replacement Algorithms

Tree-based Pseudo LRU

- Organize cache lines into a binary tree.
- Track bit at each split (k-1 bits in total)
- Update log2(k) bits upon hit and miss

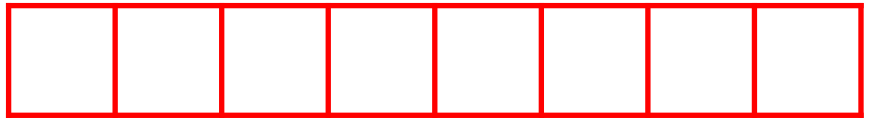
  (mark unfollowed branches as least recently used)
- Will never update one of the

  log2(k) = r-d most recently used lines



Replacement Algorithms

Most-Recently-Used Pseudo LRU



- Keep list of *k* bits (one per line)
  - On hit on line *i*, we set to 1 bit *i* (fast)
  - On miss, we replace a line whose bit is set to 0
  - If we set the last zero to one, then all the other bits back to zero
- Guarantee that unused lines are removed from the cache
- Sometimes replace a very recently used line

Memory Updates

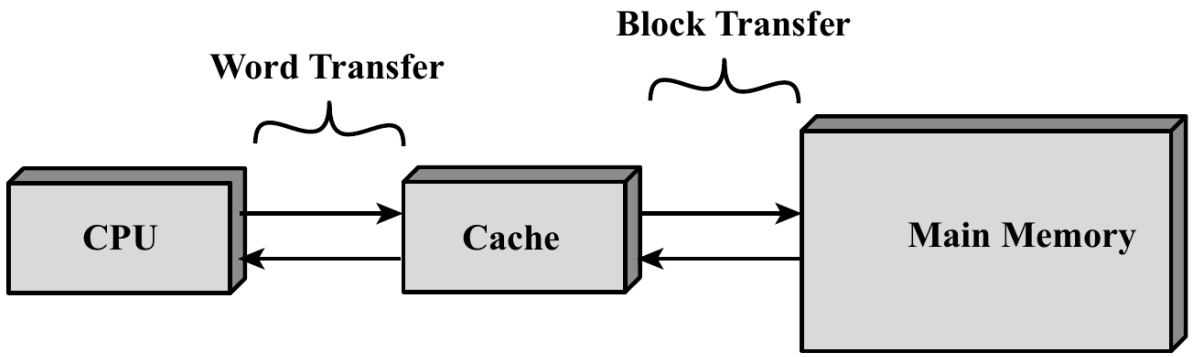
[keeping cache and main memory synchronized]

Memory Updates: Write Policy

Motivation

- Information in cache is also in main memory
- When cache gets updated, there can be discrepancies
- Problematic since other components might be able to access this information directly, without the processor.

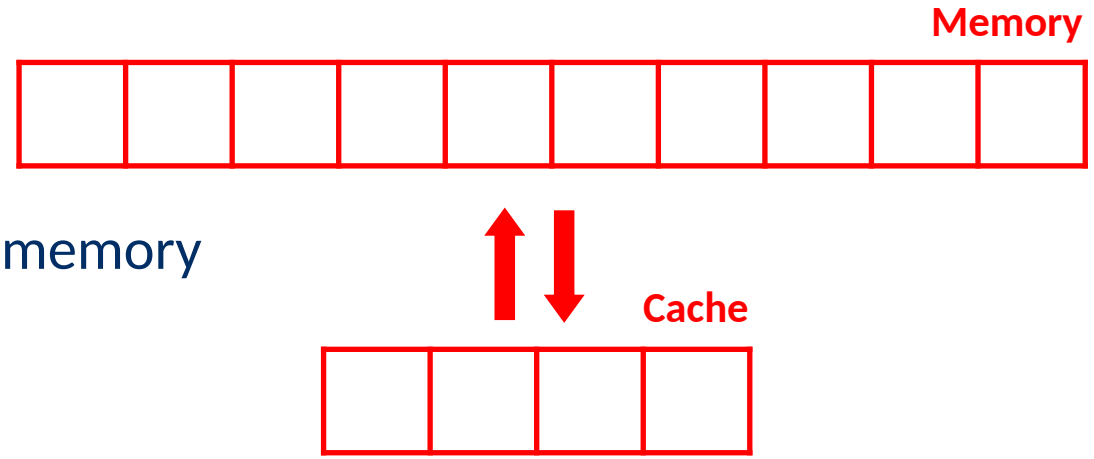
  → e.g. via direct memory access (DMA)



Memory Updates: Write Policy

Two Techniques

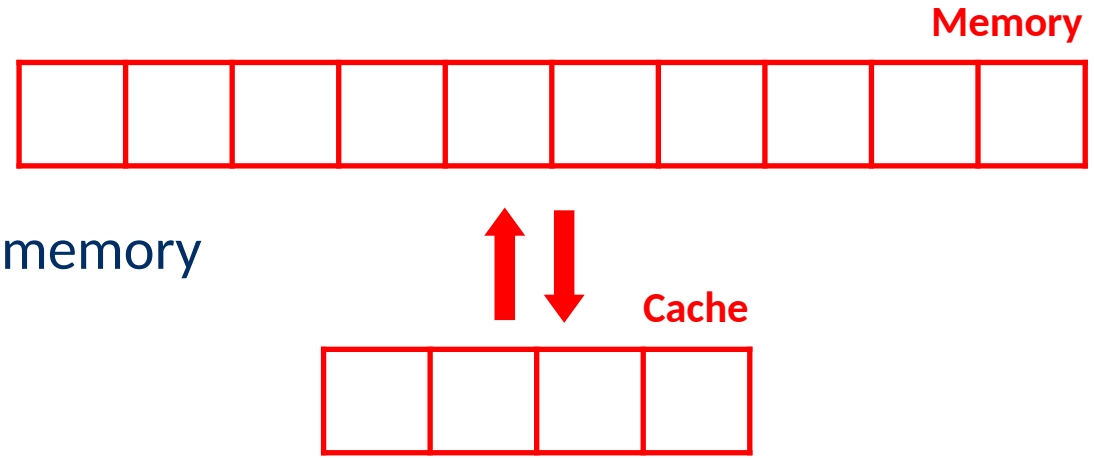
- Write through
  - Simplest but least efficient
  - Every update in cache gets pushed to the memory
  - Causes extra traffic in the data bus



Memory Updates: Write Policy

Two Techniques

- Write through
  - Simplest but least efficient
  - Every update in cache gets pushed to the memory
  - Causes extra traffic in the data bus



- Write back
  - Flag modified cache parts with an *update* bit
  - When removing a block from cache, the update bit will be checked and then the memory will be updated.

Current Designs

[how it is deployed nowadays]

Cache Memory

Current Designs

- Modern systems have multiple processors (each with its how series of cache)
  - **L1 cache**, provided with processor chip, small but very fast.
  - **L2 cache**, a bit larger than L1 but slower.
  - **L3 cache**, shared between cores, larger but slower.

    → *Inclusive* vs. *exclusive* cache
- Split cache: separate cache for instructions and data.
  - **Cons:** partial capacity lost depending on the needs
  - **Pros:** simultaneous retrieval of information.
- TLB caches → more details on virtual memory lecture.

Cache Memory

Current Designs

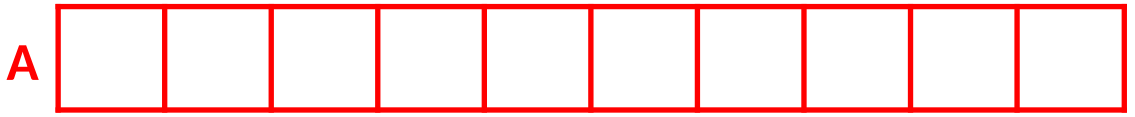
$L1$ CACHE hit	≈ 4 cycles
$L2$ CACHE hit	≈ 10 cycles
$L3$ CACHE hit	line unshared ≈ 40 cycles
$L3$ CACHE hit	shared line in another core ≈ 65 cycles
$L3$ CACHE hit	modified in another core ≈ 75 cycles
Dram	$\approx 60 - 100$ ns

Tab. 1: Intel Core i7 Xeon 5500 Series Data Source Latency

Benchmarking

[quantifying some characteristics]

Cache Benchmarking



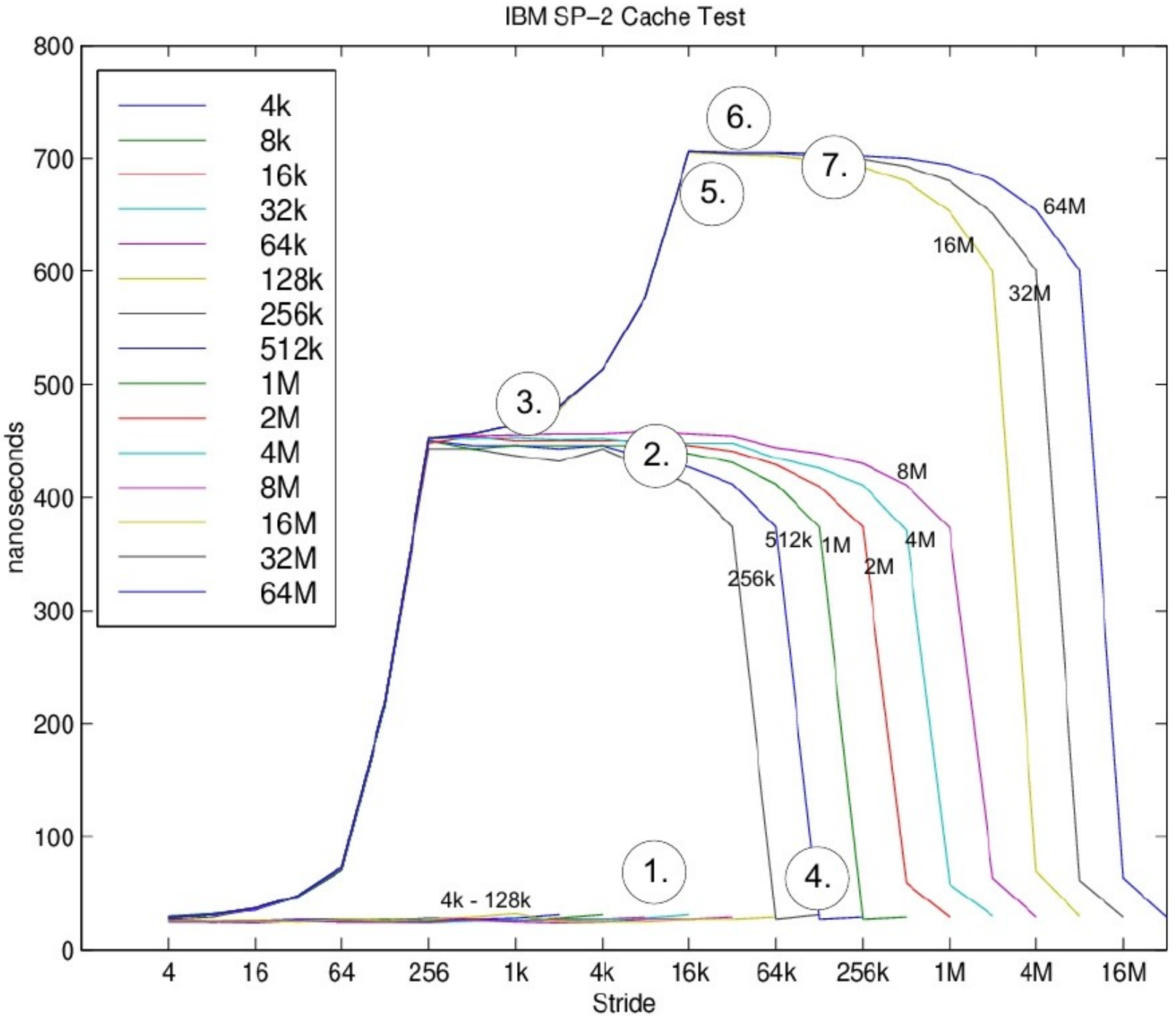
Procedure

- Repeatedly request elements from an array *A* with fixed stride 2^k
  - **k=0 → Stride 2^0:** all elements
  - **k=1 → Stride 2^1:** element 0, 2, 4, 6, etc.
  - **k=2 → Stride 2^k:** element 0, 2^k, 2.2^k, 3.2^k, etc.
- Let the stride increase from 1 element to the size of *A*
- Repeat this for different arrays whose size is doubled each time
- Measure the **average access-time** for different array **size** and **stride** values

Benchmarking

Assumption

- Replacement policy:
  FIFO or LRU



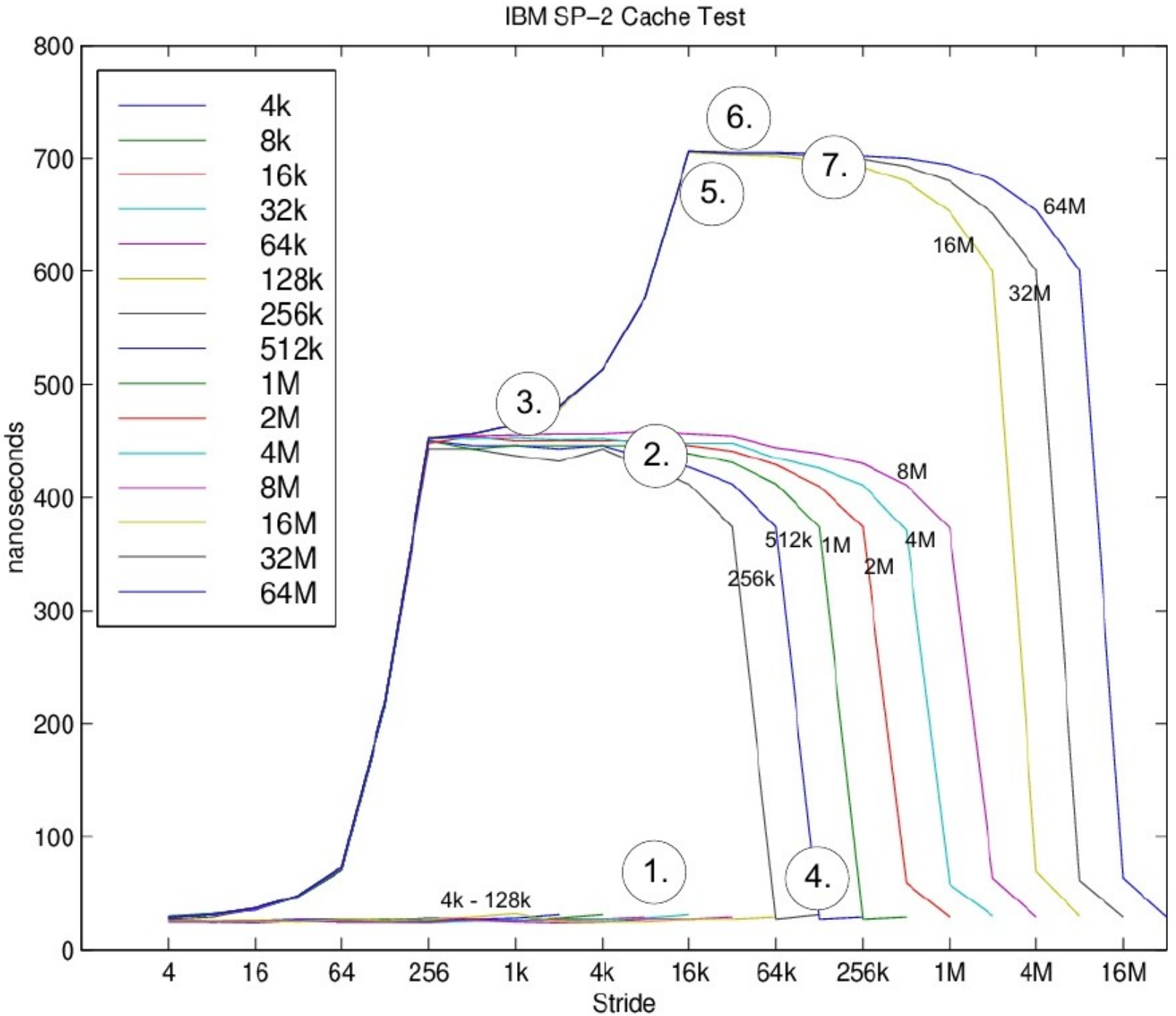
Benchmarking

Assumption

- Replacement policy:
  FIFO or LRU

Observations

1) Cache size: 128k.
2) Cache line: 256 bytes.
3) Cache 4-way associative.
4) No L2 cache in place

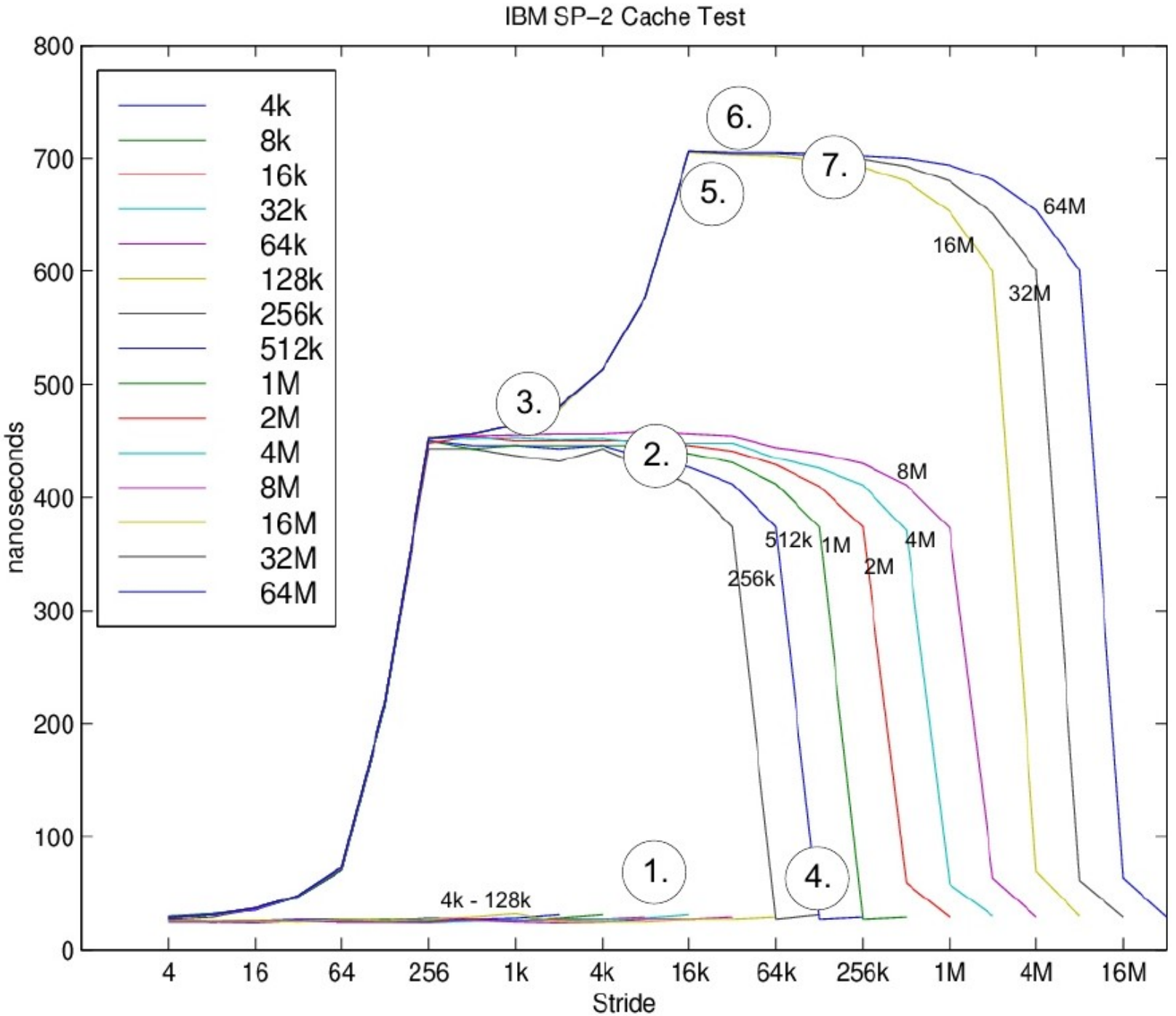


Benchmarking

Cache size: 128k

- Array size < 128K
  - very low access time
  - elements are not removed by subsequent passes

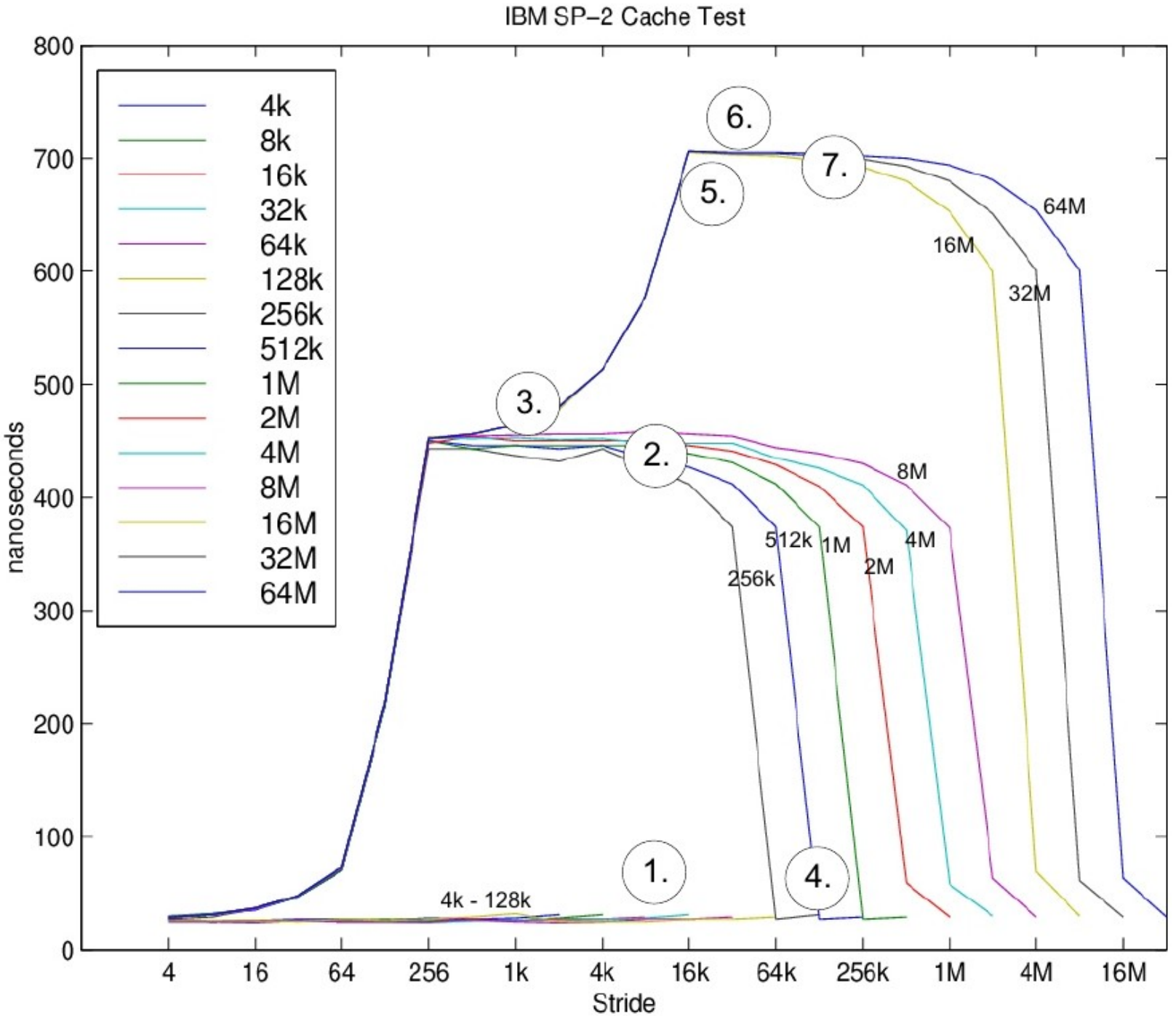
    [no misses]
- Array Size > 256K
  - Access time increases
  - Misses occur → array no longer fits in cache



Benchmarking

Cache line: 256 bytes

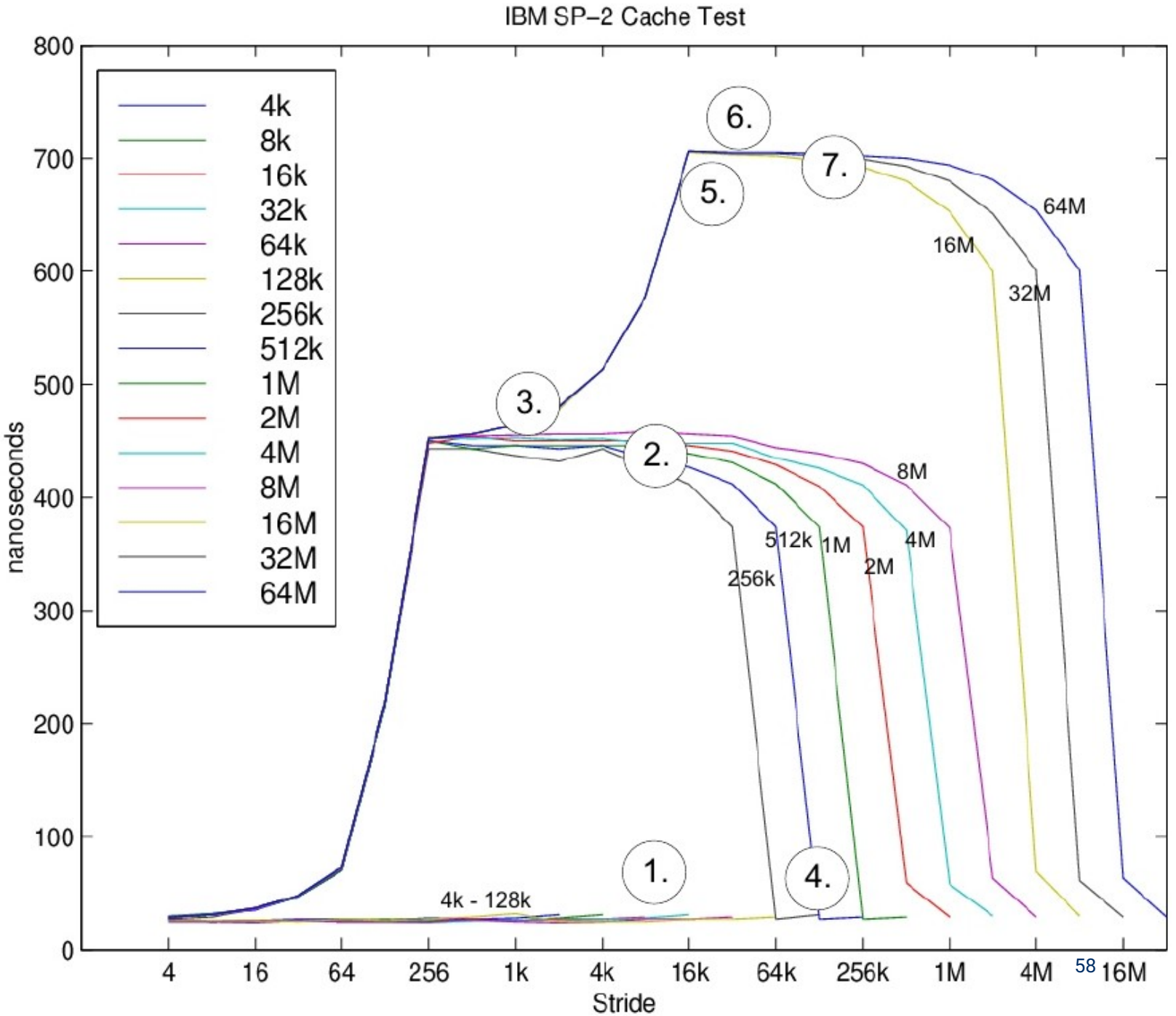
- Array size=256k, stride=128
  - access time: ~200 nsec.
- Array size=256k, stride=64
  - access time: ~75 nsec.
- Array size=256k, stride=256
  - access time: ~450 nsec.
  - Misses occur → each word belongs to a different line.



Benchmarking

Cache 4-way associative

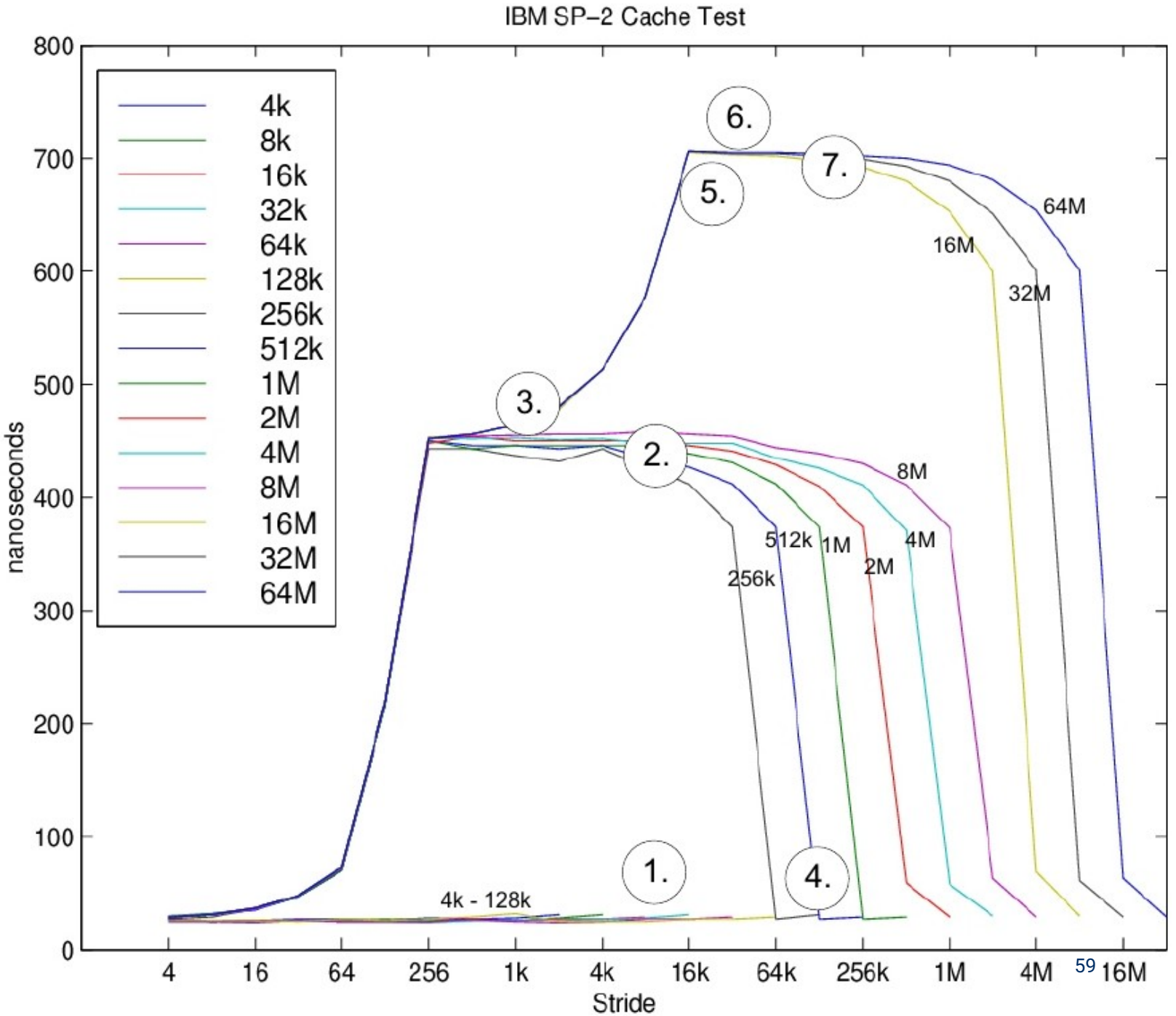
- Array size=256k, stride 128k
  - 2 elements read, mapped to the same set.
  - If only hits → set ≥ 2
- Array size=256k, stride 64k
  - 4 elements read, mapped to the same set.
  - If only hits → set ≥ 4
- Array size=256k, stride 32k
  - Access time increases
  - Misses occur → set too small for requested words



Benchmarking

No L2 cache in place

- Notice three clear levels
  - **~20 nsec.**, L1 cache hit
  - **~450 nsec.**, L1 miss, TLB cache hit
  - **~700 nsec.,** L1 and TLB miss
- If there was an L2 cache we should see an additional level



Key points

[Finally :D]

Cache Memory

Pay attention to…

- Why does the memory hierarchy work ?
- What types of mapping do we find in cache memory and to what extent are they sensitive to thrashing ?
- What are the most important replacement algorithms for caching ?

  What do you know about their performance and the complexity of their implementation ?
- How to interpret the results of benchmarking

Further Reading

Exercise Session tomorrow

- Tomorrow Room A.143 CMI, 16h00-18h00

Essential: Cache Memory

- OS Course Notes - chapter 2.
- Blackboard – Recorded lecture from Prof. Van Houdt.

Announcement

Exercise Session #1

- Tomorrow 16h00-18h00, M.G.005
- Caching + benchmarking

Questions?

Operating Systems

[1500WETOPS]

José Oramas